Linux Networking: configuración, enrutamiento y filtrado de tráfico en sistemas Linux.

Altamirano Axel, Bongiovanni Iara, Falcón Jantus Agostina, Gonzalez Ariel.

Fac. de Ciencias Exactas Naturales y Agrimensura, Universidad Nacional del Nordeste, Corrientes

Resumen: Las redes de computadoras son la base de casi cualquier servicio informático actual, y gran parte de los servidores, equipos de red y dispositivos embebidos que las sostienen funcionan con sistemas basados en Linux.
En este trabajo se analiza cómo Linux implementa y gestiona la comunicación en red. Se abordan el modelo TCP/IP dentro del kernel, la configuración de interfaces y direcciones, el enrutamiento y el filtrado de tráfico mediante Netfilter.
Como parte práctica, se recrea una topología de dos redes unidas por un router dentro de una única máquina virtual, utilizando network namespaces y las herramientas de iproute2. Las mismas pruebas se realizan en Ubuntu y en Debian, y se miden los tiempos, la memoria utilizada, la pérdida de paquetes y los puertos visibles con y sin firewall, para comparar el comportamiento de ambas distribuciones como router y firewall con nftables.

Palabras Claves: Linux, TCP/IP, iproute2, enrutamiento, Netfilter, nftables, network namespaces, Debian, Ubuntu.

Índice

1. Introducción 3
2. Marco Teórico 4
2.1 Modelo TCP/IP y encapsulamiento 4
2.2 Interfaces de red en Linux 6
2.3 Direccionamiento 7
2.4 Enrutamiento en el Kernel 9
2.5 Resolución de nombres y DHCP 10
2.6 Seguridad: Netfilter y firewalls 12
2.7 Herramientas de configuración y diagnóstico 13
2.8 Virtualización de red: network namespaces 15
2.9 Diferencias entre distribuciones: Debian y Ubuntu 16
3. Metodología 16
3.1 Entorno de pruebas 17
3.2 Herramientas utilizadas 17
3.3 Qué se mide y cómo 18
4. Desarrollo: parte práctica 18
4.1 Topología utilizada 18
4.2 Verificación de la configuración 19
4.3 Experimento 1: armado de la topología 20
4.4 Experimento 2: conectividad, enrutamiento y latencia 20
4.5 Captura de tráfico con tcpdump 23
4.6 Experimento 3: firewall con nftables 24
4.7 Experimento 4: rendimiento del router 28
4.8 Experimento 5: comparación entre Debian y Ubuntu 28
5. Resultados 29
5.1 Verificación de la configuración 29
5.2 Armado de la topología: tiempo y memoria 30
5.3 Conectividad y latencia 30
5.4 Captura de tráfico 30
5.5 Firewall 31
5.6 Rendimiento del router 31
5.7 Comparación entre Debian y Ubuntu 31
5.8 Resumen y problemas encontrados 32
6. Conclusiones 33
Referencias 34

1. Introducción

Una red puede definirse como un conjunto de nodos capaces de comunicarse entre sí. En ocasiones, esa comunicación se apoya en equipos intermedios que se encargan de transmitir los datos entre quienes los solicitan. Para que sea posible, todos los participantes deben respetar un mismo conjunto de reglas para el intercambio de mensajes, denominadas protocolos.

En la actualidad, el conjunto de protocolos **predominante es TCP/IP**, sobre el cual funcionan Internet y la mayoría de las redes locales [2].

Linux es un sistema operativo de código abierto basado en el núcleo creado por Linus Torvalds en 1991. Su arquitectura puede organizarse en tres niveles. El primero son las aplicaciones, que permiten a los usuarios realizar sus tareas. El segundo es el espacio de usuario, donde se encuentran las herramientas administrativas para configurar el sistema e instalar software. El tercero es el kernel, que gestiona los recursos y se comunica directamente con el hardware [1]. **En el kernel reside la pila de protocolos de red**: es el encargado de administrar las tarjetas de red, procesar el tráfico entrante y saliente y decidir hacia dónde se envía cada paquete.

Se entiende por Linux Networking al conjunto de herramientas, protocolos y configuraciones que permiten a un sistema Linux comunicarse con otros dispositivos, tanto en una red local como a través de Internet.

Abarca la configuración de interfaces de red, la gestión de los protocolos TCP/IP, el enrutamiento, la implementación de firewalls, la virtualización de redes y el monitoreo y diagnóstico de problemas.

Su estudio resulta relevante porque Linux está presente en buena parte de la infraestructura de red actual: servidores web y de correo, centros de datos, routers, firewalls y dispositivos IoT. Comprender cómo Linux maneja la red permite configurar y administrar estos equipos, diagnosticar fallas y protegerlos frente a ataques.

Objetivo general. El objetivo de este trabajo es analizar los principales componentes de networking en Linux y comprobar su funcionamiento de manera práctica.

Pregunta de investigación. ¿Se comportan igual Debian y Ubuntu cuando funcionan como router y firewall con Linux? Como las dos usan el kernel Linux (aunque en versiones distintas: 7.0.0-42-generic en Ubuntu y 6.12.107+deb13-amd64 en Debian), la hipótesis es que el enrutamiento, el filtrado y la velocidad dan resultados parecidos, y que las diferencias están en la forma de configurar la red, en el DNS y en los programas que vienen instalados.

Objetivos específicos.

- Armar una red de cuatro PCs y un router dentro de una sola máquina virtual, usando network namespaces.

- Medir el tiempo y la memoria que lleva armarla y verificar la conectividad entre las dos redes.

- Observar cómo viaja un paquete por el router (ARP, direcciones IP y MAC) con una captura de tráfico.

- Comprobar qué servicios ve un atacante con y sin firewall, y cómo se filtra el ping.

- Comparar Debian y Ubuntu en configuración, recursos y rendimiento.

Importancia de comparar Debian y Ubuntu. Ubuntu deriva de Debian, comparte su sistema de paquetes y usa el mismo núcleo Linux, por lo que suele suponerse que se comportan igual. Compararlas en las mismas condiciones permite separar lo que depende del kernel (enrutamiento, Netfilter, namespaces), que debería ser igual, de lo que depende de cada distribución: cómo se guarda la configuración de red, qué servicio resuelve los nombres, qué firewall viene incluido, cuántos recursos consume y cuánto tarda en aplicar los cambios. Esa información es la que necesita un administrador para elegir la distribución de un router, un firewall o un servidor, y para saber qué comandos y archivos cambian al pasar de una a otra. Además, repetir exactamente las mismas pruebas en las dos hace que los resultados sean reproducibles y comparables.

El desarrollo del trabajo se divide en el marco teórico (sección 2) y la parte práctica (sección 4). La sección 3 describe la metodología: el entorno de pruebas, las herramientas y las mediciones. La sección 5 presenta los resultados y la sección 6, las conclusiones. Cada tema del marco teórico termina con un apartado «Relación con la práctica», que indica qué prueba de la sección 4 lo comprueba.

2. Marco Teórico

2.1 Modelo TCP/IP y encapsulamiento

Las redes actuales se basan en la conmutación de paquetes. Los datos no viajan como un flujo continuo: **se dividen en pequeños bloques llamados paquetes**, que se transmiten de una máquina a otra. Esto permite que muchos usuarios compartan un mismo enlace, ya que los paquetes de cada uno se envían de forma alternada sin que la comunicación de uno interfiera con la de los demás [2].

TCP/IP no es un único protocolo, sino un conjunto de protocolos diseñados para trabajar en conjunto, cada uno definido en uno o más documentos RFC (*Request for Comments*) [1]. Sus componentes principales son:

- **IP** (*Internet Protocol*): encamina los paquetes de una máquina a otra a través de las redes [3].
- **ICMP** (*Internet Control Message Protocol*): brinda soporte de bajo nivel a IP, como mensajes de error, asistencia en el enrutamiento y ayuda para la depuración. Herramientas como **ping** y traceroute se basan en este protocolo [4].
- **ARP** (*Address Resolution Protocol*): traduce direcciones IP a direcciones de hardware (direcciones MAC). Estrictamente no forma parte de TCP/IP y puede utilizarse con otros protocolos, pero es indispensable para su funcionamiento en redes Ethernet [1], [5].

- **UDP** (*User Datagram Protocol*): envía datos sin establecer una conexión previa y sin verificar que lleguen a destino. Es más rápido y liviano, por lo que se usa en servicios como DNS [6].
- **TCP** (*Transmission Control Protocol*): establece conversaciones confiables, bidireccionales (*full duplex*), con control de flujo y corrección de errores [7].

Estos protocolos se organizan convencionalmente en un modelo de cinco capas: física, de enlace, de red, de transporte y de aplicación [1]. Cada capa resuelve un problema distinto y se apoya en la inferior, como muestra la Figura 1. El hardware se maneja en la capa de enlace, por lo que los protocolos de niveles superiores no necesitan conocer el tipo de red física utilizada (Ethernet cableada, Wi-Fi, fibra óptica, etc.).

Los datos viajan en forma de paquetes, cuyo tamaño máximo lo impone la capa de enlace. Cada paquete está formado por una cabecera, que indica su origen y destino y puede incluir información de control como sumas de verificación (*checksums*), y una carga útil, que son los datos a transferir [1]. La unidad de datos recibe un nombre distinto según la capa: en la capa de enlace se llama trama (*frame*), en la capa de red, paquete, y en la capa de transporte, segmento en el caso de TCP o datagrama en el caso de UDP.

Cuando un sistema envía datos, estos recorren la pila de protocolos de arriba hacia abajo, y cada capa agrega su propia cabecera. El paquete completo de una capa se convierte así en la carga útil de la capa inferior. Este proceso se denomina encapsulamiento. En la máquina que recibe el paquete ocurre el proceso inverso: cada capa retira su cabecera a medida que los datos suben por la pila [1]. La Figura 2 muestra un ejemplo: 100 bytes de datos de una aplicación se convierten en un paquete UDP de 108 bytes, luego en un paquete IPv4 de 128 bytes y finalmente en una trama Ethernet de 146 bytes.

En Linux, las capas de red y de transporte están implementadas dentro del kernel. Las aplicaciones, que se ejecutan en el espacio de usuario, acceden a ellas mediante la interfaz de *sockets*. Un socket representa un extremo de la comunicación: la aplicación lo crea indicando el protocolo de transporte (TCP o UDP), y luego lo asocia a una dirección y un puerto locales para escuchar conexiones, o se conecta a una dirección y un puerto remotos [1]. De este modo, un programa puede comunicarse por la red sin ocuparse de cómo los datos se encapsulan y transmiten.

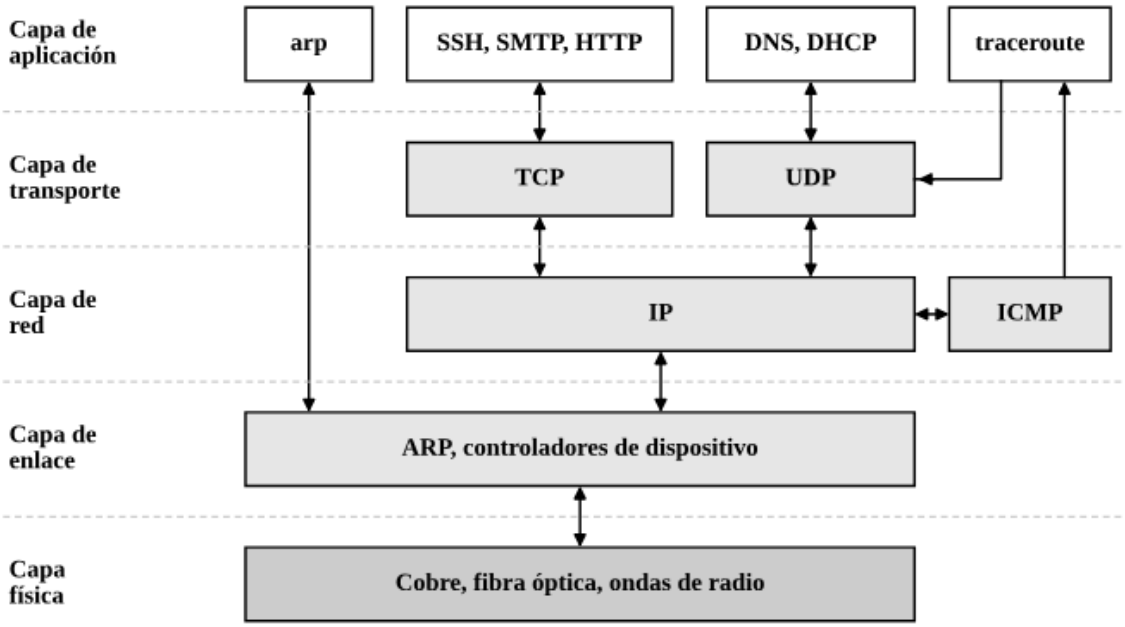


Figura 1. Modelo de capas de TCP/IP. Fuente: adaptado de [1].

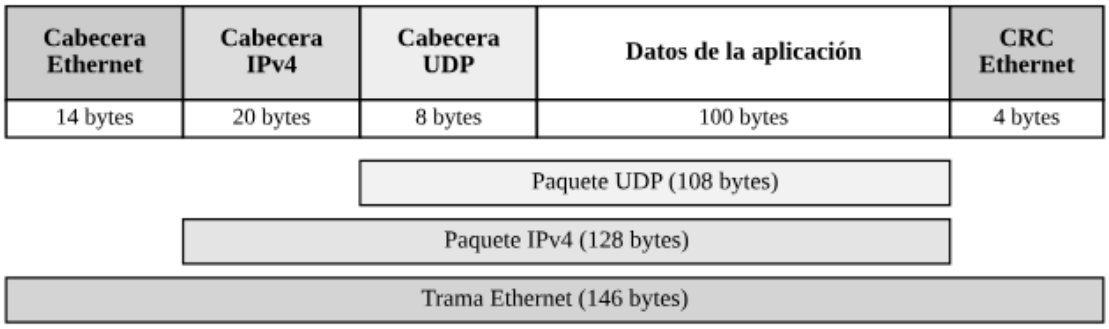


Figura 2. Encapsulamiento de un paquete UDP típico. Fuente: adaptado de [1].

Relación con la práctica. El modelo se observa de forma directa en las pruebas: ping y traceroute usan ICMP [4], [21] (sección 4.4), la captura con tcpdump [21] muestra el ARP [5] que resuelve la MAC del siguiente salto y los paquetes ICMP que le siguen (sección 4.5), y el escaneo con nmap [32] detecta puertos TCP abiertos en el equipo de destino (sección 4.6).

2.2 Interfaces de red en Linux

Una interfaz de red es el punto, físico o virtual, a través del cual los datos entran y salen de un sistema Linux. Cada interfaz tiene un nombre, puede tener una o más direcciones IP asignadas y, en el caso de las interfaces Ethernet o inalámbricas, posee una dirección MAC que la identifica dentro de la red física [1]. Es importante destacar que las direcciones IP se asignan a las interfaces y no a las máquinas: un equipo con varias interfaces tendrá una dirección distinta en cada una de ellas.

Las interfaces pueden clasificarse en tres grandes grupos.

Interfaces físicas. Corresponden a una tarjeta de red real. La más común es Ethernet, la tecnología predominante en redes locales. En sus primeras versiones, Ethernet utilizaba un único cable coaxial compartido al que se conectaban todos los nodos. Hoy en día se implementa con una topología en estrella: cada equipo se conecta mediante cable de par trenzado o fibra óptica a un switch, que reenvía cada trama sólo al puerto de destino.

Esto permite comunicación *full duplex* y velocidades de 100 Mbps, 1 Gbps o superiores [2]. Una trama Ethernet puede transportar hasta 1500 bytes de datos; este valor se conoce como MTU (*Maximum Transmission Unit*). También son interfaces físicas las inalámbricas (Wi-Fi), que cumplen la misma función sin cables.

Interfaz de loopback. Es una interfaz virtual que existe en todo sistema Linux y se denomina lo. Tiene asignada la dirección 127.0.0.1 y permite que el sistema se comunique consigo mismo: los paquetes enviados a ella nunca salen a la red. Se utiliza, por ejemplo, para que un programa se conecte a un servicio que corre en el mismo equipo [1].

Interfaces virtuales. Son creadas por software dentro del kernel y no dependen de ningún hardware. Entre ellas se encuentran los pares *veth* (*virtual Ethernet*), que funcionan como un cable virtual con dos extremos: todo lo que entra por uno sale por el otro. También existen los *bridges* (puentes), que conectan varias interfaces entre sí y se comportan como un switch virtual [8]. Este tipo de interfaces es la base de la virtualización de redes en Linux, utilizada por máquinas virtuales, contenedores y, en este trabajo, por la parte práctica.

En cuanto a los nombres, tradicionalmente las interfaces Ethernet se llamaban **eth0, eth1**, etc., según el orden en que el kernel las detectaba. Ese orden podía cambiar entre reinicios, por lo que las distribuciones actuales utilizan nombres predecibles basados en la ubicación física del hardware. Por ejemplo, **enp0s3** es una interfaz Ethernet (*en*) en el bus **PCI 0**, ranura 3, y **wlp2s0** es una interfaz inalámbrica (*wl*) [9].

Las interfaces de un sistema pueden consultarse con el comando ip link show, que lista cada una con su estado (activa o inactiva), su MTU y su dirección MAC [8]. Su configuración se trata con más detalle en la sección 2.7.

Relación con la práctica. Cada PC, switch y router de la topología tiene su propia interfaz de loopback y sus interfaces virtuales veth, que se comprueban con **ip addr show** [21] (sección 4.2). La interfaz dummy0 del Experimento 5 es otro ejemplo de interfaz virtual (sección 4.8).

2.3 Direccionamiento

Para que un paquete llegue a su destino debe estar correctamente direccionado. En TCP/IP se combinan varios esquemas de direccionamiento: las direcciones MAC, utilizadas por el hardware; las direcciones IP (IPv4 e IPv6), utilizadas por el software; y los nombres de host, pensados para las personas [1]. Estos últimos se tratan en la sección 2.5.

Direcciones MAC. Cada interfaz de red física posee una dirección MAC (*Media Access Control*) de 48 bits, que se escribe como seis pares de dígitos hexadecimales, por ejemplo 08:00:27:3a:5c:91. En las interfaces físicas, los tres primeros bytes identifican al fabricante de la tarjeta. En las interfaces virtuales, como los pares veth, la MAC la genera el kernel y es una dirección administrada localmente (con el segundo bit del primer byte en 1), como 02:fb:f5:96:86:b7 o 6a:88:12:c1:c7:f3 en la práctica. Esta dirección distingue a la interfaz dentro de la red física local y es la que utiliza la capa de enlace para entregar las tramas [1].

Direcciones IPv4. A diferencia de las MAC, las direcciones IP son independientes del hardware. Una dirección IPv4 tiene 32 bits (4 bytes) y se representa con cuatro números decimales entre 0 y 255 separados por puntos, como **192.168.1.10**. Toda dirección se divide en una porción de red, que identifica a una red lógica, y una porción de host, que identifica a una interfaz dentro de esa red [1]. La correspondencia entre direcciones IP y direcciones MAC dentro de una red local la resuelve el protocolo ARP.

No siempre es correcto decir que las direcciones IP son únicas a nivel global. Existen rangos reservados para redes privadas (10.0.0.0/8, 172.16.0.0/12 y 192.168.0.0/16) que pueden usarse en muchas redes a la vez, ya que no son visibles desde Internet [10]. Para que esos equipos accedan a Internet se utiliza NAT (*Network Address Translation*), que permite que varias máquinas compartan una única dirección pública [1].

Tipos de direcciones. La capa de red define varios tipos de direcciones según su destino [1]:

- *Unicast*: identifica a una única interfaz de red.
- *Multicast*: identifica simultáneamente a un grupo de hosts.
- *Broadcast*: incluye a todos los hosts de una subred local.
- *Anycast*: identifica a un grupo de hosts, pero el paquete se entrega a solo uno de ellos, generalmente el más cercano.

Clases de direcciones. Originalmente, el límite entre la porción de red y la de host se determinaba según el valor del primer byte de la dirección, que definía su clase (Tabla 1). Este esquema resultó poco flexible y hoy se considera histórico, aunque sigue siendo una referencia común [1].

Clase	Primer byte	Formato	Uso
A	1-127	R.H.H.H	Redes muy grandes
B	128-191	R.R.H.H	Redes medianas
C	192-223	R.R.R.H	Redes pequeñas
D	224-239	-	Direccionamiento Multicast
E	240-255	-	Direcciones Experimentales

R: byte de red; H: byte de host. El valor 0 no se utiliza como primer byte, y la red 127 está reservada para loopback.

Subnetting y máscaras. Para aprovechar mejor las direcciones, parte de la porción de host puede reasignarse a la porción de red, dividiendo una red en subredes más pequeñas. Esto se indica con una

máscara de subred de 32 bits, donde los bits en 1 marcan la porción de red. Por ejemplo, la máscara 255.255.255.0 indica que los primeros 24 bits corresponden a la red [1]. En una red 192.168.1.0 con esa máscara hay 256 direcciones, de las cuales 254 son asignables a hosts: la primera (192.168.1.0) identifica a la red y la última (192.168.1.255) es la de broadcast.

CIDR. El direccionamiento entre dominios sin clases (*Classless Inter-Domain Routing*) abandona por completo el esquema de clases y define el tamaño de la porción de red solamente mediante la máscara, que se escribe con la notación de prefijo: 192.168.1.0/24 equivale a la máscara 255.255.255.0 [11]. CIDR permite crear redes de cualquier tamaño. Por ejemplo, la red /24 anterior puede dividirse en dos subredes /25: 192.168.1.0/25 (hosts .1 a .126) y 192.168.1.128/25 (hosts .129 a .254). También permite agrupar varias redes bajo un único prefijo más corto, lo que simplifica las tablas de enrutamiento [1], [11]. En Linux, las direcciones se asignan directamente en esta notación, por ejemplo con **ip addr add 192.168.1.10/24 dev enp0s3.**

Direcciones IPv6. Ante el agotamiento de las direcciones IPv4, se desarrolló IPv6, cuyas direcciones tienen 128 bits (16 bytes). Se escriben como ocho grupos de cuatro dígitos hexadecimales separados por dos puntos, como **2001:0db8:0000:0000:0000:0000:0000:0001**, que puede abreviarse **2001:db8::1**. Generalmente, los primeros 64 bits corresponden a la red y los últimos 64 identifican a la interfaz [1]. Linux soporta IPv4 e IPv6 de forma simultánea en una misma interfaz.

Relación con la práctica. La topología usa dos redes privadas, 192.168.1.0/24 y 192.168.2.0/24 (sección 4.1). La captura con tcpdump [21] permite comprobar que las direcciones IP de origen y destino no cambian de punta a punta, mientras que las direcciones MAC cambian en cada red (sección 4.5).

2.4 Enrutamiento en el Kernel

El enrutamiento (*routing*) es el proceso de dirigir un paquete a través del conjunto de redes que separan a su origen de su destino [1]. Cada equipo de la red no necesita conocer el camino completo, sino solo el siguiente salto (*next hop*) hacia el destino.

El término enrutamiento se usa para dos tareas distintas: buscar en la tabla de ruteo hacia dónde enviar un paquete, y construir esa tabla en primer lugar [1]. En Linux, la primera la realiza siempre el kernel; la segunda puede hacerse de forma manual o automática, como se explicará más adelante.

La tabla de ruteo. La información de enrutamiento se almacena en una tabla dentro del kernel. Cada entrada indica una red de destino con su máscara, la interfaz de salida y, si corresponde, el *gateway* o puerta de enlace: el router al que se debe entregar el paquete para alcanzar esa red [1]. La tabla puede consultarse con el comando **ip route show** [12]. Un resultado típico es el siguiente:

default via 192.168.1.1 dev enp0s3

192.168.1.0/24 dev enp0s3 proto kernel scope link src 192.168.1.10

La segunda línea es una ruta directamente conectada: el kernel la crea automáticamente al asignar la dirección **192.168.1.10/24** a la interfaz, e indica que los hosts de esa red se alcanzan directamente, sin pasar por un router.

La primera es la ruta por defecto (*default*, equivalente a `0.0.0.0/0`), que se utiliza para cualquier destino que no coincida con otra entrada; en este caso, esos paquetes se envían al gateway **192.168.1.1**, normalmente el router que conecta la red local con Internet.

Elección de la ruta. Cuando debe enviar un paquete, el kernel compara la dirección de destino con todas las entradas de la tabla y, entre las que coinciden, elige la más específica, es decir, la de máscara más larga (*longest prefix match*) [1]. Por ejemplo, si la tabla contiene una ruta hacia **10.0.0.0/8** y otra hacia **10.1.0.0/16**, un paquete con destino **10.1.2.3** coincide con ambas, pero se envía por la segunda, que es más específica. La ruta por defecto, con prefijo /0, es la menos específica posible y solo se usa como último recurso. Si ninguna entrada coincide y no existe ruta por defecto, el destino se considera inalcanzable y el paquete se descarta.

Una vez elegida la ruta, el kernel debe entregar la trama al siguiente salto dentro de la red local. Para ello utiliza ARP y obtiene la dirección MAC del gateway (o del propio destino, si está en la misma red). Así, la dirección IP de destino del paquete se mantiene a lo largo de todo el camino, mientras que las direcciones MAC cambian en cada salto.

Linux como router. Por defecto, un sistema Linux se comporta como un host: procesa los paquetes dirigidos a él y descarta los que no lo son. Sin embargo, puede funcionar como router reenviando paquetes entre sus interfaces. Para ello basta con activar el parámetro del kernel **net.ipv4.ip_forward (con sysctl -w net.ipv4.ip_forward=1)** [13]. Esta capacidad es la que permite que muchos routers, firewalls y equipos de red funcionen con Linux, y es la que se utiliza en la parte práctica de este trabajo.

Rutas estáticas y dinámicas. Las rutas pueden agregarse manualmente, por ejemplo con **ip route add 192.168.2.0/24 via 192.168.1.1**, lo que se denomina enrutamiento estático [12]. En redes grandes o cambiantes se utiliza enrutamiento dinámico: protocolos como RIP, OSPF o BGP permiten que los routers intercambien información y actualicen sus tablas automáticamente. En Linux, estos protocolos no se ejecutan en el kernel, sino en programas del espacio de usuario que calculan las rutas y las cargan en la tabla del kernel [1]. Así, la decisión de cómo construir la tabla queda separada del reenvío de los paquetes, que siempre realiza el kernel.

Relación con la práctica. Es el apartado que más se vincula con las pruebas. En el Experimento 2 (sección 4.4) se ve que pc1 no puede alcanzar la otra red sin una ruta por defecto, que con la ruta pero sin reenvío (**ip_forward** en 0) [13] el router descarta los paquetes, y que al activarlo el ping funciona y las respuestas llegan con TTL 63: salen de pc3 con TTL 64, el valor por defecto en Linux [13], y el router les resta una unidad [3]. El traceroute muestra los dos saltos del recorrido.

2.5 Resolución de nombres y DHCP

Las direcciones IP son secuencias de números difíciles de recordar para las personas. Por eso, los sistemas operativos permiten asociar uno o más nombres de host (*hostnames*) a una dirección IP, de modo que los usuarios puedan escribir un nombre como **www.unne.edu.ar** en lugar de su dirección [1]. El proceso de traducir un nombre a su dirección IP se denomina resolución de

nombres. En Linux puede realizarse de dos formas: mediante un archivo local o mediante consultas a servidores DNS.

El archivo /etc/hosts. Es la forma más simple de resolución: un archivo de texto donde cada línea asocia una dirección IP con uno o más nombres [14]. Por ejemplo:

127.0.0.1 localhost
192.168.1.20 servidor-web

Es útil en redes pequeñas o para pruebas, pero no escala: cada equipo debe tener su propia copia actualizada. El nombre del propio equipo, por su parte, se guarda en el archivo **/etc/hostname.**

DNS. El sistema de nombres de dominio (*Domain Name System*) resuelve este problema mediante una base de datos jerárquica y distribuida entre miles de servidores en todo el mundo [15]. Cuando un equipo necesita resolver un nombre, envía una consulta (normalmente por UDP, al puerto 53) a un servidor DNS, generalmente provisto por el router local o por el proveedor de Internet [2]. En Linux, los servidores a consultar se indican en el archivo /etc/resolv.conf, mediante líneas como nameserver 8.8.8.8 [14].

En distribuciones actuales como Ubuntu, la resolución la gestiona el **servicio systemd-resolved**. En ese caso, /etc/resolv.conf apunta a la dirección local 127.0.0.53, donde este servicio recibe las consultas, las reenvía a los servidores DNS reales y guarda las respuestas en caché para acelerar consultas futuras [17]. En Debian con escritorio, en cambio, systemd-resolved no está activo y NetworkManager escribe directamente en /etc/resolv.conf los servidores DNS que recibe por DHCP (sección 5.7).

Orden de consulta. El archivo /etc/nsswitch.conf define en qué orden se consultan las fuentes. La línea habitual hosts: files dns indica que primero se busca en /etc/hosts y, si el nombre no está allí, se consulta al DNS [14]. Por eso, una entrada en /etc/hosts tiene prioridad sobre la respuesta del DNS. Para verificar la resolución pueden usarse herramientas como dig, nslookup o getent hosts.

DHCP. Para que un equipo funcione en la red necesita una dirección IP, una máscara, un gateway y al menos un servidor DNS. Configurar estos datos a mano en cada equipo sería poco práctico, por lo que se utiliza DHCP (*Dynamic Host Configuration Protocol*), que los asigna automáticamente [16]. El proceso tiene cuatro pasos, conocidos como DORA: el cliente envía un mensaje *Discover* por broadcast buscando un servidor; el servidor responde con una oferta (*Offer*); el cliente la solicita (*Request*); y el servidor la confirma (*Acknowledge*). La dirección se otorga por un tiempo limitado, llamado *lease* o concesión, que el cliente debe renovar periódicamente. En Linux, el cliente DHCP suele estar integrado en el gestor de red del sistema, como NetworkManager.

Relación con la práctica. DHCP no se prueba en la parte práctica. La resolución de nombres se compara entre distribuciones mirando el archivo /etc/resolv.conf y el servicio de DNS que usa cada una [14], [17] (sección 4.8).

2.6 Seguridad: Netfilter y firewalls

Un firewall es un mecanismo que controla el tráfico de red, permitiendo o bloqueando paquetes según un conjunto de reglas definidas por el administrador. Su objetivo es proteger un equipo o una red frente a accesos no autorizados, limitando qué servicios son accesibles y desde dónde [1]. En Linux, esta función no depende de un programa externo, sino que está integrada en el propio kernel a través de Netfilter.

Netfilter. Es el componente del kernel Linux que permite inspeccionar, filtrar y modificar los paquetes a medida que atraviesan la pila de red [18]. Para ello define una serie de puntos de enganche (*hooks*) en el recorrido de cada paquete, donde se evalúan las reglas configuradas:

- ***PREROUTING***: apenas llega el paquete, antes de decidir hacia dónde se envía.
- ***INPUT***: para los paquetes cuyo destino es el propio equipo.
- ***FORWARD***: para los paquetes que el equipo reenvía de una interfaz a otra, cuando funciona como router.
- ***OUTPUT***: para los paquetes generados por el propio equipo.
- ***POSTROUTING***: justo antes de que el paquete salga por una interfaz.

Así, un paquete que llega al equipo pasa por PREROUTING y, según la decisión de enrutamiento, continúa por INPUT (si es para el equipo) o por FORWARD y POSTROUTING (si debe reenviarse). Un paquete generado localmente pasa por OUTPUT y POSTROUTING (Figura 3).

Reglas y acciones. Cada regla está formada por condiciones de coincidencia, como la dirección de origen o destino, el protocolo o el puerto, y una acción que se aplica a los paquetes que las cumplen. Las acciones más comunes son aceptar el paquete (*accept*), descartarlo en silencio (*drop*) o rechazarlo avisando al emisor (*reject*). Si un paquete no coincide con ninguna regla, se le aplica la política por defecto de esa cadena [18].

Netfilter permite además un filtrado con estado (*stateful*): mediante el seguimiento de conexiones (*connection tracking*), el kernel recuerda las conexiones activas y puede distinguir un paquete que inicia una conexión nueva de uno que responde a una ya establecida. Esto permite, por ejemplo, bloquear todas las conexiones entrantes y aun así recibir las respuestas a las conexiones iniciadas desde el propio equipo [18]. Netfilter también es el mecanismo que implementa NAT en Linux, mencionado en la sección 2.3.

Herramientas de configuración. Netfilter se configura desde el espacio de usuario con distintas herramientas:

- *iptables*: la herramienta clásica, utilizada durante muchos años. Organiza las reglas en tablas (como *filter* para filtrado y *nat* para traducción de direcciones) y cadenas que corresponden a los *hooks* anteriores. Requiere herramientas separadas para IPv4, IPv6 y otros protocolos [18].
- *nftables*: su sucesor, incorporado al kernel en 2014 y utilizado por defecto en las distribuciones actuales. Unifica en un solo comando, nft, el manejo de IPv4, IPv6 y otros protocolos, con una sintaxis más simple y consistente [19].
- *ufw* (*Uncomplicated Firewall*): una interfaz simplificada, incluida en Ubuntu, que permite crear reglas comunes con comandos sencillos, como **ufw allow 22/tcp** para permitir conexiones SSH. Internamente traduce esas órdenes a reglas de Netfilter [20].

Como ejemplo, las siguientes reglas de nftables crean una tabla y una cadena asociada al *hook* FORWARD, y descartan los pedidos de ping (ICMP *echo-request*) que provengan de la red 192.168.2.0/24 y que el equipo deba reenviar:

nft add table inet filtro
nft add chain inet filtro forward '{ type filter hook forward priority 0; policy accept; }'
nft add rule inet filtro forward ip saddr 192.168.2.0/24 icmp type echo-request drop

Una regla de este tipo se aplica en la parte práctica de este trabajo (sección 4).

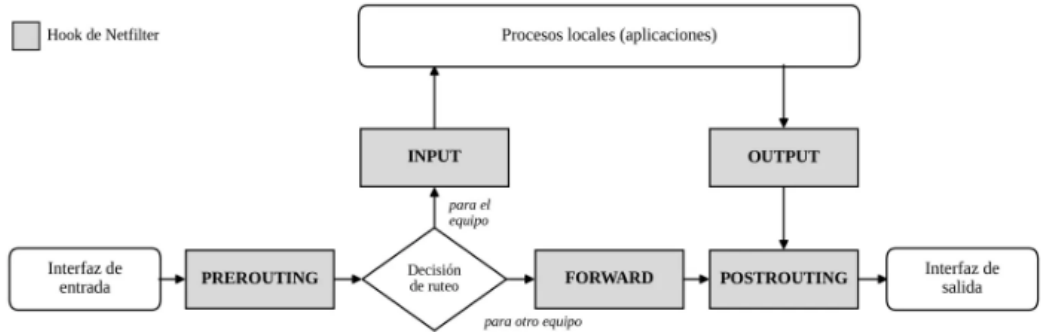


Figura 3. Recorrido de un paquete por los *hooks* de Netfilter. Fuente: elaboración propia a partir de [18].

Relación con la práctica. Se carga una regla de nftables [19] en el hook FORWARD del router. Con ella se bloquea el ping que sale de la red B, se cuentan los paquetes descartados con un contador y se compara el escaneo de puertos con nmap con y sin firewall (sección 4.6). También se verifica si ufw [20], [31] viene instalado o activo en cada distribución (sección 4.8).

2.7 Herramientas de configuración y diagnóstico

Linux ofrece un conjunto de herramientas de línea de comandos para configurar la red y diagnosticar problemas. Durante muchos años se utilizó el paquete *net-tools*, que incluye comandos como **ifconfig, route, arp y netstat**. Este paquete se considera obsoleto y fue reemplazado por ***iproute2***, que agrupa la mayoría de las funciones en un único comando, ip, y aprovecha mejor las capacidades actuales del kernel [22]. En distribuciones recientes como Ubuntu, net-tools ya no viene instalado por defecto. La Tabla 2 muestra la equivalencia entre ambos.

Tarea	**net-tools (obsoleto)**	**iproute2 (actual)**
Ver interfaces y direcciones	ifconfig	ip addr show
Activar o desactivar una interfaz	ifconfig enp0s3 up	ip link set enp0s3 up

Ver tabla de ruteo	route -n	ip route show
Ver la tabla de ARP	arp -n	ip neigh show
Ver conexión	netstat -tulpn	ss -tulpn

Tabla 2. Equivalencias entre net-tools e iproute2. Fuente: elaboración propia a partir de [21], [22].

El comando ip. Se organiza en subcomandos según el objeto sobre el que se trabaja: **ip link** para las interfaces, **ip addr** para las direcciones IP, **ip route** para la tabla de ruteo e **ip neigh** para la tabla de vecinos, es decir, la caché de ARP [21]. Por ejemplo, **ip addr add 192.168.1.10/24 dev enp0s3** asigna una dirección a una interfaz, e **ip route add default via 192.168.1.1** define la ruta por defecto. Es importante tener en cuenta que los cambios hechos con **ip** se aplican de inmediato, pero son temporales: se pierden al reiniciar el equipo.

Herramientas de diagnóstico.

- **ping**: envía mensajes ICMP *echo-request* a un destino y espera las respuestas (*echo-reply*). Permite comprobar si un equipo es alcanzable y medir el tiempo de ida y vuelta de los paquetes [21].
- **traceroute**: muestra los routers por los que pasa un paquete hasta llegar a su destino. Para ello envía paquetes, en general UDP, con un valor de TTL (*Time To Live*) creciente: cada router descuenta uno a ese valor y, cuando llega a cero, descarta el paquete y avisa al emisor con un mensaje ICMP. Así, cada router del camino se identifica a sí mismo [1]. Este funcionamiento es el que muestran las flechas de traceroute en la Figura 1.
- ss: lista los *sockets* abiertos en el sistema, es decir, las conexiones activas y los puertos en escucha, junto con el proceso que los utiliza [21].
- **tcpdump**: captura y muestra los paquetes que pasan por una interfaz, con sus cabeceras. Permite filtrar por protocolo, dirección o puerto, por ejemplo **tcpdump -i enp0s3 icmp**, y guardar la captura en un archivo que luego puede abrirse con Wireshark para analizarla de forma gráfica [21].

Configuración persistente. Para que la configuración se mantenga después de reiniciar, se guarda en archivos que el sistema aplica al arrancar. Distintas distribuciones utilizan distintas herramientas. NetworkManager, presente en la mayoría de los sistemas de escritorio y en Red Hat, puede configurarse desde la línea de comandos con nmcli [23]. Ubuntu utiliza Netplan: la configuración se describe en archivos YAML dentro de /etc/netplan/, y Netplan la traduce para NetworkManager o para systemd-networkd, el gestor de red de systemd. Debian, en cambio, utiliza tradicionalmente el paquete ifupdown, que lee la configuración del archivo /etc/network/interfaces. En la práctica se comprobó que, con escritorio, Ubuntu guarda las conexiones creadas con nmcli como archivos de Netplan (/etc/netplan/90-NM-<uuid>.yaml), mientras que Debian usa el formato keyfile de NetworkManager (/etc/NetworkManager/system-connections/<nombre>.nmconnection).

Relación con la práctica. Todos los comandos de la sección 4 pertenecen a este apartado: **ip**, **ping**, **traceroute**, **ss**, **tcpdump** y **nft** [19], [21]. La configuración persistente se prueba en el Experimento 5, creando una interfaz que debe mantenerse después de reiniciar (sección 4.8) [24], [27].

2.8 Virtualización de red: network namespaces

La virtualización de redes consiste en crear, mediante software, elementos de red que funcionan como si fueran físicos: interfaces, switches, routers e incluso redes completas. Linux ofrece esta capacidad de forma nativa a través de los *namespaces* (espacios de nombres).

Un *namespace* es una función del kernel que permite aislar un conjunto de recursos del sistema, de modo que los procesos que se ejecutan dentro de él solo ven sus propios recursos y no los del resto del equipo [25]. Existen distintos tipos de namespaces según el recurso que aíslan, como los procesos, los sistemas de archivos montados o los usuarios. El que interesa a este trabajo es el *network namespace*, que aísla la pila de red.

Cada network namespace tiene su propia copia independiente de todos los elementos vistos en las secciones anteriores: sus interfaces de red (incluida su propia interfaz de loopback), sus direcciones IP, su tabla de ruteo, su tabla ARP y sus reglas de Netfilter [25]. En la práctica, un namespace se comporta como un equipo separado con su propia configuración de red, aunque comparte el mismo kernel que el resto del sistema. Al iniciar Linux, todos los procesos se ejecutan en un namespace inicial, que es el que contiene las interfaces físicas del equipo.

Los namespaces están aislados entre sí, por lo que para comunicarlos se utilizan las interfaces virtuales descritas en la sección 2.2. Un par *veth* funciona como un cable virtual: colocando cada extremo en un namespace distinto, ambos quedan conectados como si fueran dos equipos unidos por un cable. Si se necesita conectar varios namespaces a una misma red, sus extremos pueden unirse a un *bridge*, que cumple la función de un switch.

Los namespaces se administran con el subcomando ip netns [25]. Por ejemplo, los siguientes comandos crean un namespace llamado pc1, lo conectan mediante un par veth y le asignan una dirección IP:

ip netns add pc1

ip link add pc1-eth0 type veth peer name sw1-eth1

ip link set pc1-eth0 netns pc1

ip netns exec pc1 ip addr add 192.168.1.10/24 dev pc1-eth0

ip netns exec pc1 ip link set pc1-eth0 up

El comando **ip netns exec** permite ejecutar cualquier programa dentro de un namespace. Así, **ip netns exec pc1 ping 192.168.1.1** realiza un ping desde **pc1**, usando sus propias interfaces y su propia tabla de ruteo.

Esta tecnología es la base de herramientas muy difundidas. Los contenedores, como los que gestiona Docker, utilizan namespaces para que cada contenedor tenga su propia red aislada [1]. Mininet, el emulador de redes usado frecuentemente para experimentar con redes definidas por software (SDN), representa cada host de la red emulada con un network namespace [26]. Su principal ventaja frente a las máquinas virtuales es que son muy livianos: permiten emular una topología completa, con varios

equipos y routers, dentro de un único sistema Linux y en pocos segundos. Por este motivo se utilizan en la parte práctica de este trabajo.

Relación con la práctica. Toda la topología de la parte práctica está hecha con network namespaces: son siete (pc1 a pc4, r1, sw1 y sw2), unidos con pares veth y con bridges que cumplen el rol de switches (sección 4.1) [25].

2.9 Diferencias entre distribuciones: Debian y Ubuntu

Ubuntu está basada en Debian [30], y ambas comparten el sistema de paquetes (dpkg y apt) y el kernel Linux, aunque cada una lo distribuye en su propia versión (en este trabajo, 7.0.0-42-generic en Ubuntu y 6.12.107+deb13-amd64 en Debian). Por eso las herramientas de las secciones anteriores (ip, nft, tcpdump) deberían funcionar igual en las dos. Las diferencias están en la capa de administración que cada distribución trae configurada por defecto, y son las que se comparan en este trabajo (Tabla 3).

Aspecto	**Ubuntu**	**Debian**
Red en instalaciones sin escritorio	Netplan (archivos YAML en /etc/netplan/) [24]	ifupdown (archivo /etc/network/interfaces) [27], [28]
Red con escritorio	NetworkManager, administrado a través de Netplan [24]	NetworkManager [29]
Resolución de nombres	systemd-resolved (servidor local 127.0.0.53) [17]	Archivo /etc/resolv.conf tradicional, salvo que se instale otro servicio [14], [27] (comprobado: lo escribe NetworkManager y systemd-resolved está inactivo)
Firewall incluido	ufw, desactivado por defecto [31]	Ninguno: ufw no viene instalado y el ruleset de nftables está vacío (verificado en el Experimento 5)
Versión utilizada en este trabajo	Ubuntu 24.04.4 LTS	Debian 13 (trixie)

Tabla 3. Diferencias esperadas en la configuración de red entre Ubuntu y Debian. Fuente: elaboración propia a partir de [14], [17], [24] y [27] a [31].

En este trabajo las dos distribuciones se instalan con escritorio, por lo que ambas utilizan NetworkManager. **Relación con la práctica.** El Experimento 5 (sección 4.8) comprueba qué archivos de configuración y qué servicios existen realmente en cada una.

3. Metodología

La metodología consiste en armar una red de prueba dentro de una máquina virtual, realizar sobre ella una serie de experimentos y repetirlos en dos distribuciones, Ubuntu y Debian, para comparar los resultados. Para cada experimento se define un resultado esperado, a partir del marco teórico, y se lo contrasta con el obtenido.

3.1 Entorno de pruebas

Se utilizan dos máquinas virtuales con los mismos recursos, una con Ubuntu y otra con Debian, de modo que las diferencias que aparezcan se deban a la distribución y no al equipo. Ambas se instalan con escritorio. Las máquinas virtuales se crean con VirtualBox [34]. La única diferencia es el disco: 25 GB en Ubuntu y 20 GB en Debian (Tabla 4), que no influye en las pruebas de red.

Dato	Ubuntu	Debian
Versión	Ubuntu 24.04.4 LTS (Desktop)	Debian GNU/Linux 13 (trixie) con GNOME
Kernel	7.0.0-42-generic	6.12.107+deb13-amd64
Memoria RAM	4 GB (4096 MB; 3911 MB visibles)	4 GB (4096 MB; 3921 MB visibles)
Procesadores	2	2
Disco	25 GB (dinámico)	20 GB (dinámico). Diferencia: la VM de Debian se creó con 20 GB
Virtualizador y red	VirtualBox 7.0.20, adaptador puente (Wi-Fi) [34]	VirtualBox 7.0.20, adaptador puente (Wi-Fi) [34]

Tabla 4. Características de las máquinas virtuales utilizadas. Fuente: elaboración propia.

3.2 Herramientas utilizadas

Las herramientas son las descriptas en la sección 2.7, más nft (sección 2.6), nmap e iperf3. La Tabla 5 indica para qué se usa cada una y en qué parte de la práctica.

Herramienta	Para qué se usa	Sección	Referencia
ip, ip netns	Crear, consultar y configurar la topología	4.1 y 4.2	[21], [25]
sysctl	Activar el reenvío de paquetes en el router	4.4	[13]
ping, traceroute	Medir conectividad, pérdida de paquetes, latencia y saltos	4.4	[4], [21]
tcpdump	Capturar el tráfico que pasa por el router	4.5	[21]
nft	Cargar reglas de firewall y leer el contador de paquetes descartados	4.6	[19]
nmap	Escanear los puertos abiertos de una PC	4.6	[32]
iperf3	Medir la velocidad de transferencia a través del router	4.7	[33]
time, free	Medir tiempos de ejecución y memoria utilizada	4.3 y 4.8	–

Tabla 5. Herramientas utilizadas en la parte práctica. Fuente: elaboración propia.

3.3 Qué se mide y cómo

Cada medición se repite cinco veces y se informa el promedio. Los valores medidos son los siguientes:

- El tiempo de ejecución de los scripts de armado y de limpieza, con el comando **time**.
- La memoria utilizada antes y después de armar la topología, con **free -m**.
- La pérdida de paquetes y la latencia (mínima, promedio y máxima) con **ping -c 20**, y la cantidad de saltos con **traceroute**.
- La cantidad de paquetes ARP e ICMP capturados con **tcpdump**.
- Los puertos abiertos y filtrados con **nmap**, y los paquetes descartados por el contador de **nft**.
- La velocidad de transferencia, en Mbit/s, con **iperf3**.

Todos los experimentos se realizan con los mismos comandos en las dos distribuciones. La topología es temporal: si la máquina virtual se reinicia, desaparece y hay que armarla de nuevo.

4. Desarrollo: parte práctica

En esta sección se describe cada prueba, los comandos utilizados y el resultado que se espera. Los comandos se ejecutan como root. Los resultados medidos se presentan en la sección 5.

4.1 Topología utilizada

La topología tiene dos redes, A (192.168.1.0/24) y B (192.168.2.0/24), cada una con dos PCs y un switch, unidas por un router (Figura 4). Todo se arma dentro de una única máquina virtual: cada PC, cada switch y el router es un network namespace (siete en total: pc1 a pc4, r1, sw1 y sw2), los cables son pares veth y los switches son bridges (sección 2.8) [25].

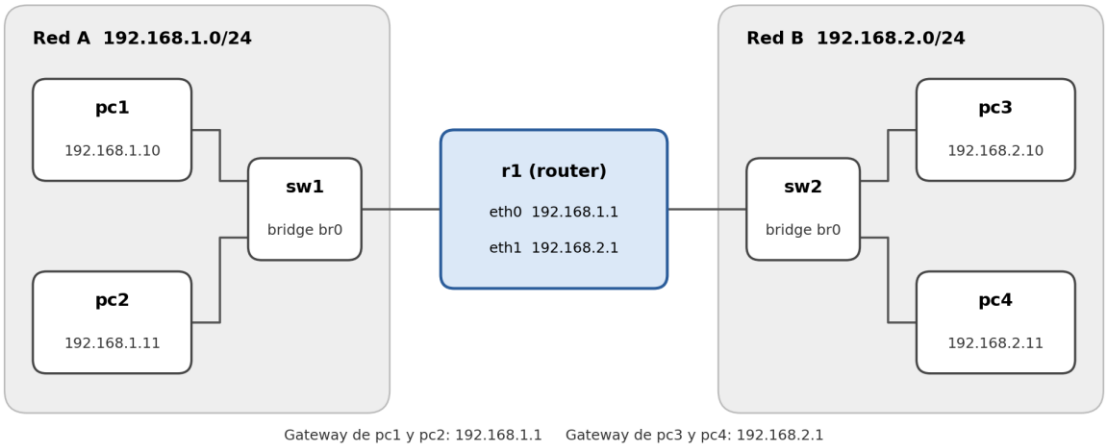


Figura 4. Topología utilizada: dos redes unidas por un router. Fuente: elaboración propia.

Equipo	Interfaz	Dirección IP	Gateway
pc1	pc1-eth0	192.168.1.10/24	192.168.1.1
pc2	pc2-eth0	192.168.1.11/24	192.168.1.1
pc3	pc3-eth0	192.168.2.10/24	192.168.2.1
pc4	pc4-eth0	192.168.2.11/24	192.168.2.1
r1	r1-eth0	192.168.1.1/24	–
r1	r1-eth1	192.168.2.1/24	–

Tabla 6. Direcciones IP de la topología. Fuente: elaboración propia.

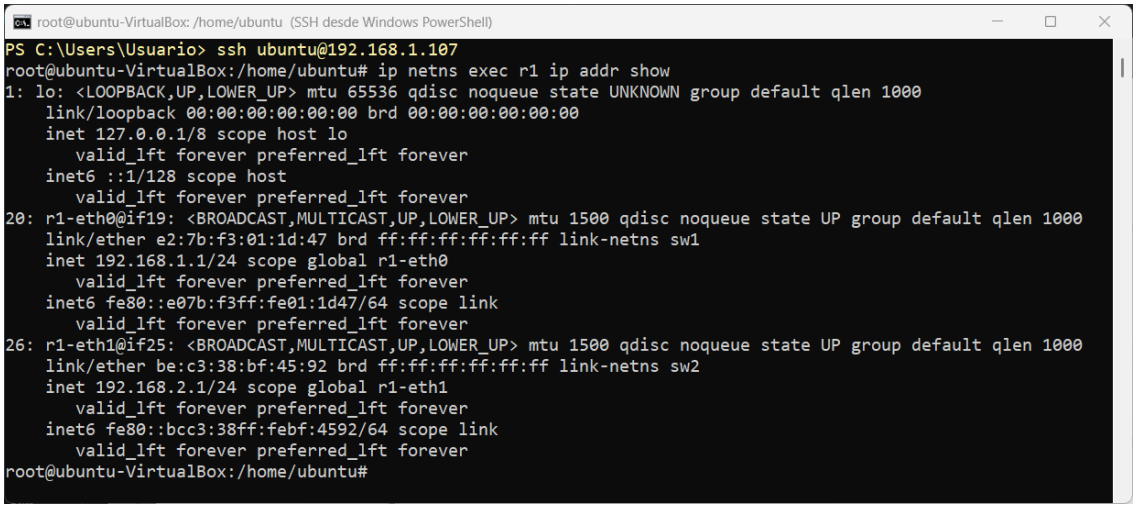
La topología se crea con el script **armar_topologia.sh** y se borra con **limpiar.sh**. El script crea un namespace por equipo, los conecta con pares veth, arma un bridge en sw1 y en sw2 y asigna las direcciones IP. No carga rutas ni activa el reenvío del router: eso se hace a mano en el Experimento 2, porque es parte de lo que se quiere mostrar.

4.2 Verificación de la configuración

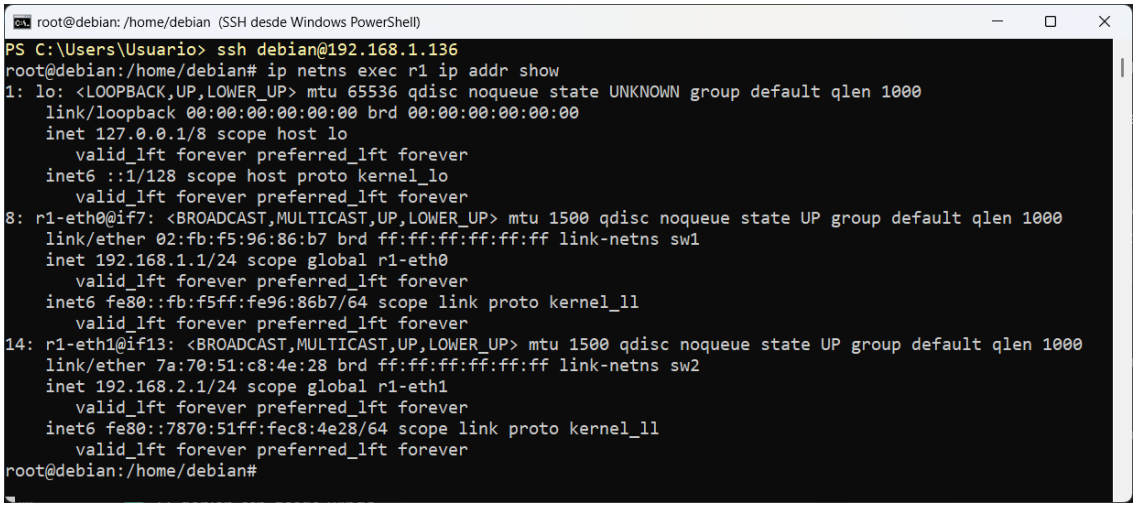
Después de armar la topología se comprueba que cada equipo tiene sus interfaces y sus direcciones, y se anotan las direcciones MAC.

ip netns list
ip netns exec pc1 ip addr show
ip netns exec r1 ip addr show

Se espera que haya siete namespaces. En **pc1** debe haber dos interfaces, **lo** y **pc1-eth0** con 192.168.1.10/24. En **r1** debe haber tres: **lo**, **r1-eth0** con 192.168.1.1/24 y **r1-eth1** con 192.168.2.1/24. Cada interfaz muestra su dirección MAC en la línea **link/ether** (sección 2.2).



(a) Ubuntu 24.04.4



(b) Debian 13

Figura 5. Interfaces y direcciones de r1: (a) Ubuntu, (b) Debian. Fuente: elaboración propia.

4.3 Experimento 1: armado de la topología

Se mide cuánto tarda el script en armar y en limpiar la topología, y cuánta memoria se utiliza.

free -m
time bash armar_topologia.sh
free -m
time bash limpiar.sh

El ciclo se repite cinco veces y se calcula el promedio. Como los namespaces son livianos (sección 2.8), se espera un tiempo corto y una diferencia de memoria pequeña. Los resultados están en la sección 5.2.

4.4 Experimento 2: conectividad, enrutamiento y latencia

Se prueba la comunicación entre las PCs en cuatro etapas, que ponen en práctica lo explicado en la sección 2.4.

ip netns exec pc1 ping -c 20 192.168.1.11
ip netns exec pc1 ping -c 4 192.168.2.10
ip netns exec pc1 ip route show
ip netns exec pc1 ip route add default via 192.168.1.1
ip netns exec r1 sysctl net.ipv4.ip_forward
ip netns exec pc1 ping -c 4 192.168.2.10
ip netns exec r1 sysctl -w net.ipv4.ip_forward=1
ip netns exec pc1 ping -c 20 192.168.2.10
ip netns exec pc1 traceroute -n 192.168.2.10

La ruta por defecto se agrega en las cuatro PCs, cada una con el gateway de su red. Se espera lo siguiente:

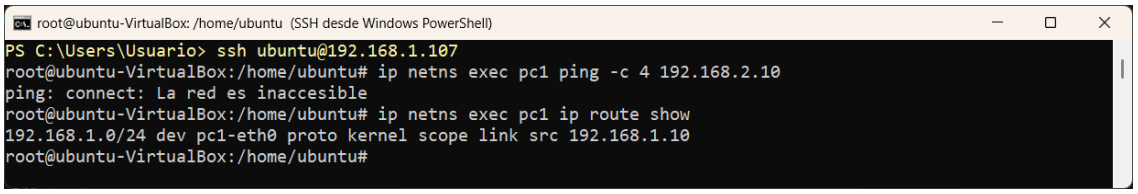
- **Misma red:** el ping funciona, sin pérdida de paquetes.

- **Otra red, sin rutas:** falla con el mensaje Network is unreachable (en un sistema en español: «ping: connect: La red es inaccesible»), porque pc1 solo conoce su propia red.

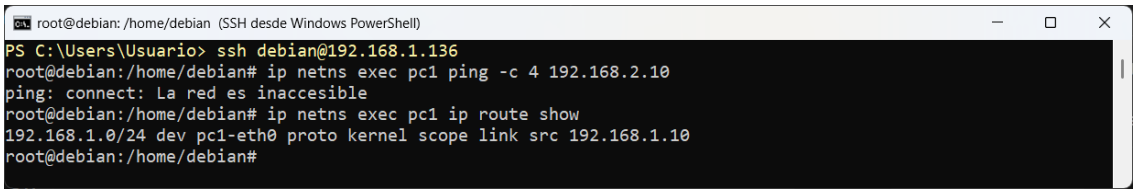
- **Con rutas pero sin reenvío:** el ping falla con 100% de pérdida, porque pc1 envía los paquetes al router pero este los descarta (**ip_forward** vale 0) [13].

- **Con el reenvío activado:** el ping funciona y las respuestas llegan con TTL 63: salen de pc3 con TTL 64, el valor por defecto en Linux [13], y el router les resta una unidad [3].

- **Traceroute:** muestra dos saltos, el router y el destino.

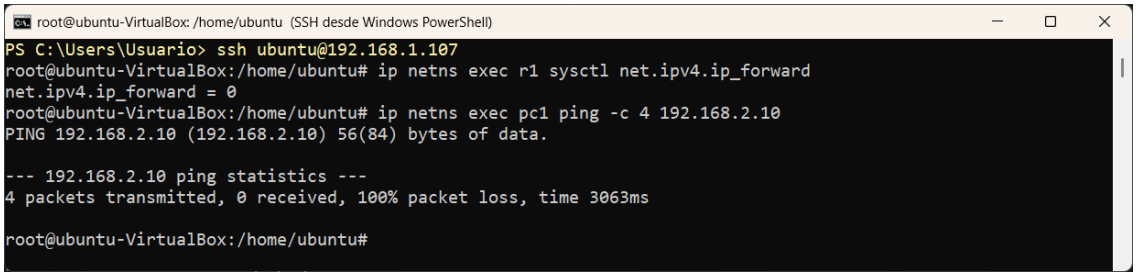


(a) Ubuntu 24.04.4

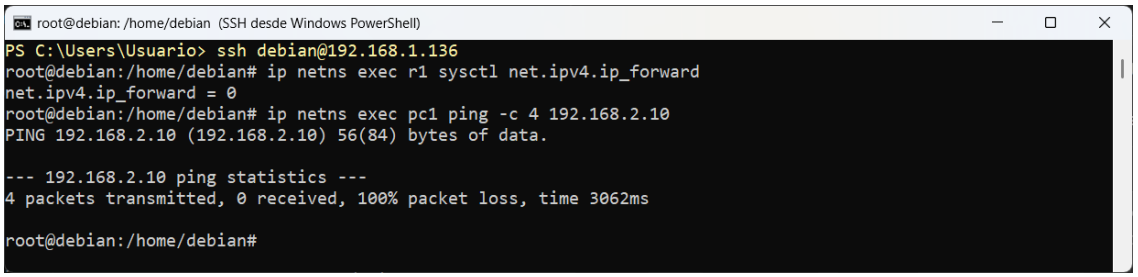


(b) Debian 13

Figura 6. Ping sin rutas y tabla de ruteo de pc1: (a) Ubuntu, (b) Debian. Fuente: elaboración propia.

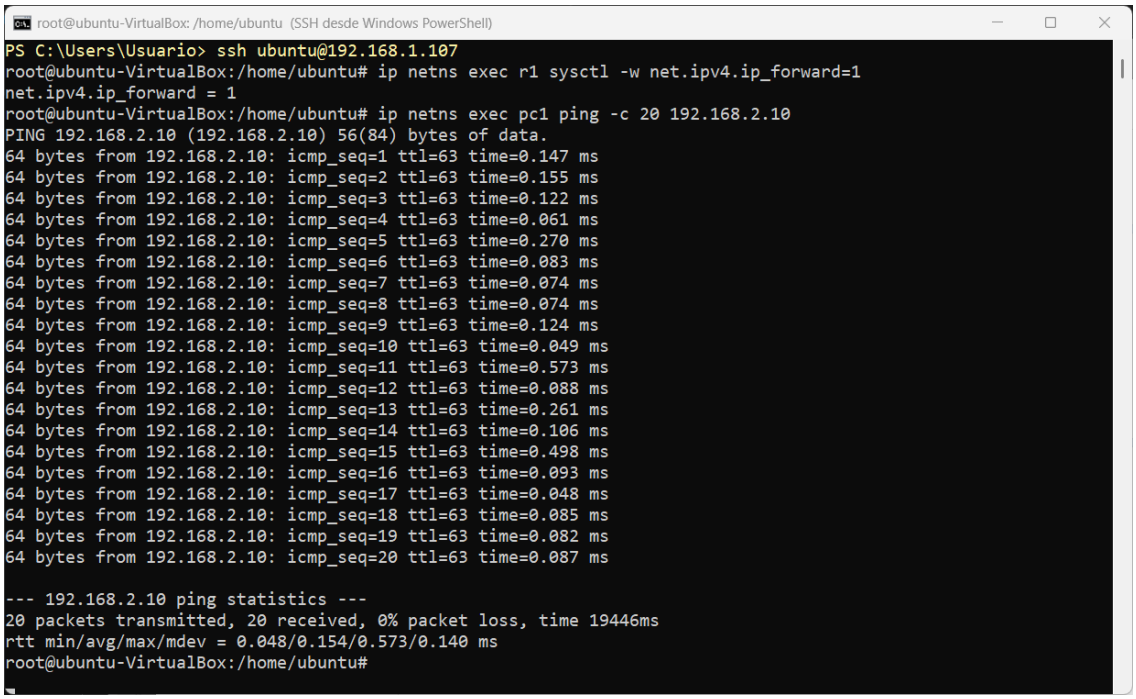


(a) Ubuntu 24.04.4

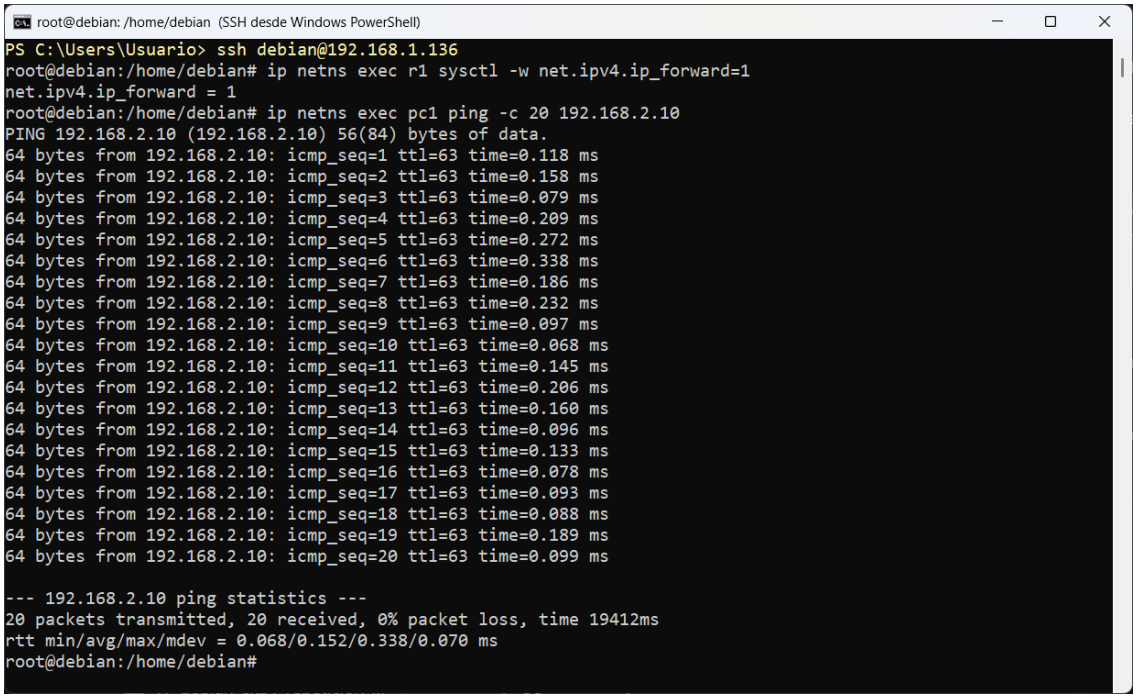


(b) Debian 13

Figura 7. Ping con rutas y con el reenvío desactivado: (a) Ubuntu, (b) Debian. Fuente: elaboración propia.

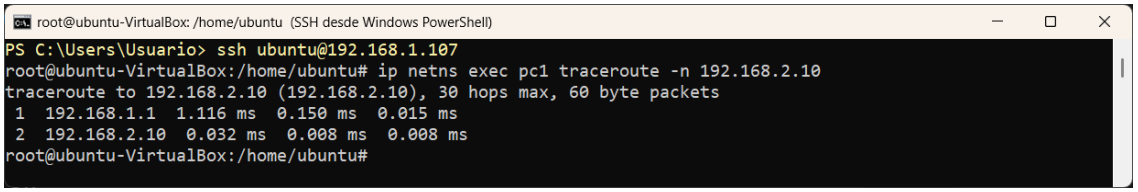


(a) Ubuntu 24.04.4

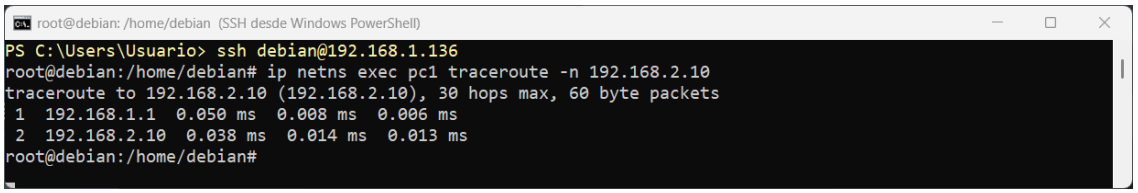


(b) Debian 13

Figura 8. Ping con el reenvío activado: (a) Ubuntu, (b) Debian. Fuente: elaboración propia.



(a) Ubuntu 24.04.4



(b) Debian 13

Figura 9. Traceroute de pc1 a pc3: (a) Ubuntu, (b) Debian. Fuente: elaboración propia.

4.5 Captura de tráfico con tcpdump

Se captura el tráfico del router mientras pc1 hace ping a pc3, para ver el ARP y los mensajes ICMP, y comprobar que las direcciones IP se mantienen mientras las MAC cambian en cada red (sección 2.4). Se usan dos terminales.

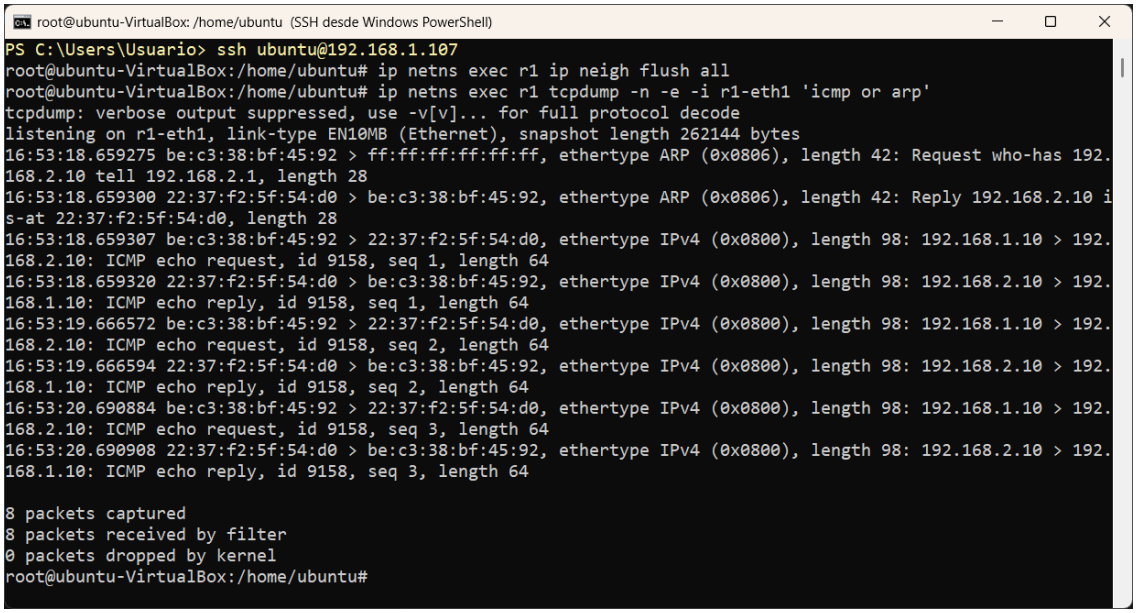
Terminal 1:

ip netns exec r1 ip neigh flush all
ip netns exec r1 tcpdump -n -e -i r1-eth1 'icmp or arp'

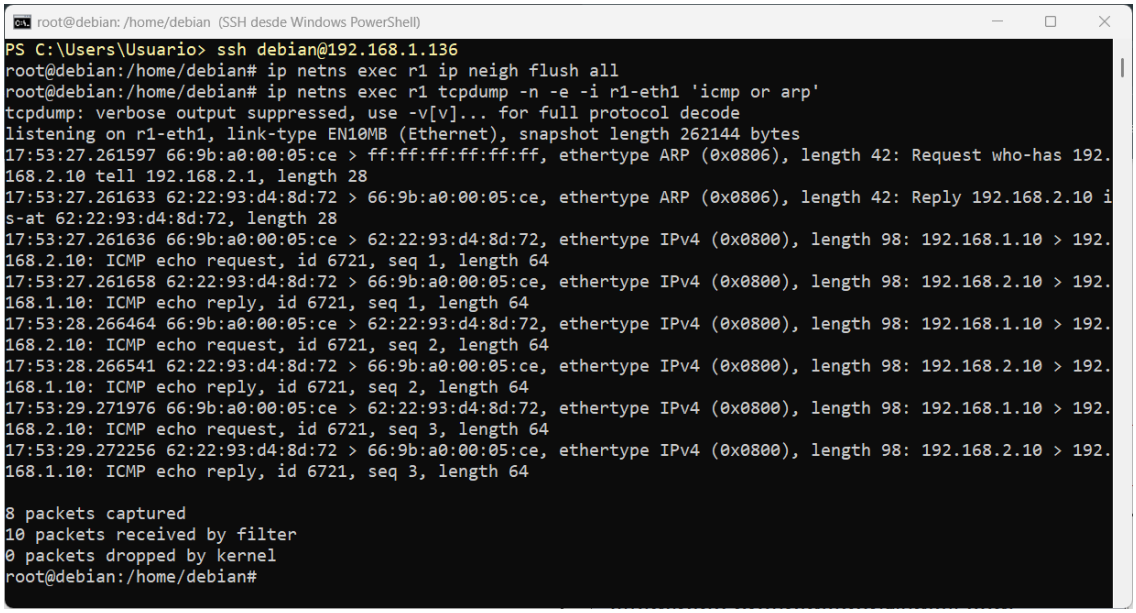
Terminal 2:

ip netns exec pc1 ping -c 3 192.168.2.10

Al terminar el ping se detiene la captura con Ctrl + C. Se espera ver primero un pedido ARP (who-has 192.168.2.10 tell 192.168.2.1) y su respuesta, y después tres pedidos y tres respuestas de ping. La opción **-e** muestra las direcciones MAC de cada trama [21]. Luego se repite la captura con **-i r1-eth0** para comparar: las direcciones IP son las mismas y las MAC son distintas. En r1-eth0 el orden cambia: el primer echo-request de pc1 aparece antes del ARP, porque r1 recién pregunta por la MAC de pc1 cuando tiene que enviarle la respuesta.



(a) Ubuntu 24.04.4



(b) Debian 13

Figura 10. Captura de tráfico en la interfaz r1-eth1: (a) Ubuntu, (b) Debian. Fuente: elaboración propia.

4.6 Experimento 3: firewall con nftables

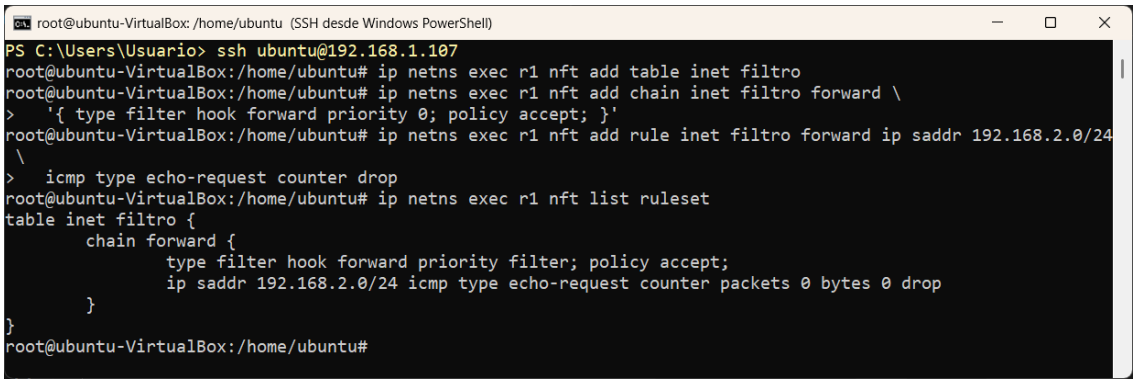
Se prueban dos usos de una regla de firewall en el hook FORWARD del router (sección 2.6) [19].

a) Bloqueo del ping de la red B. La regla descarta los pedidos de ping (echo-request) cuyo origen es la red B y cuenta los paquetes descartados.

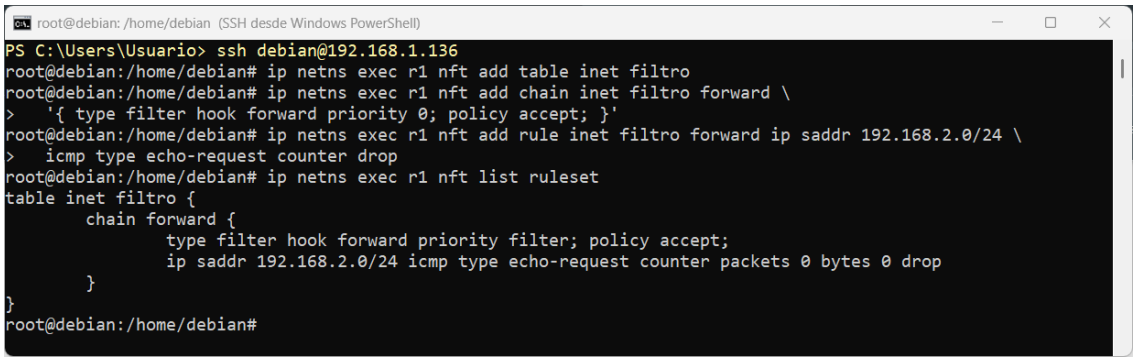
ip netns exec r1 nft add table inet filtro

ip netns exec r1 nft add chain inet filtro forward '{ type filter hook forward priority 0; policy accept; }'
ip netns exec r1 nft add rule inet filtro forward ip saddr 192.168.2.0/24 icmp type echo-request counter drop
ip netns exec r1 nft list ruleset
ip netns exec pc3 ping -c 4 192.168.1.10
ip netns exec pc1 ping -c 4 192.168.2.10
ip netns exec r1 nft delete table inet filtro

Se espera que el ping de pc3 a pc1 falle con 100% de pérdida, y que el de pc1 a pc3 funcione, porque las respuestas de pc3 son echo-reply y la regla no las bloquea. El contador debería marcar cuatro paquetes descartados, uno por cada ping de pc3.

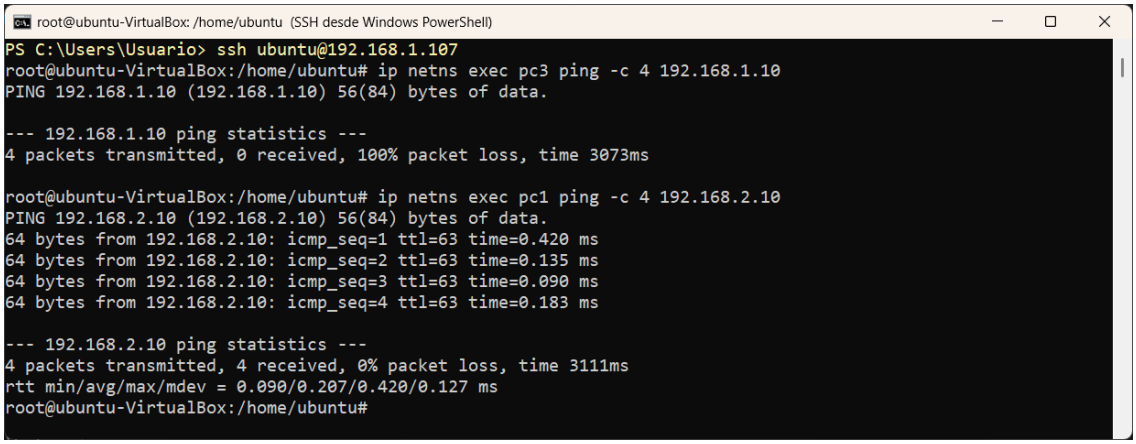


(a) Ubuntu 24.04.4

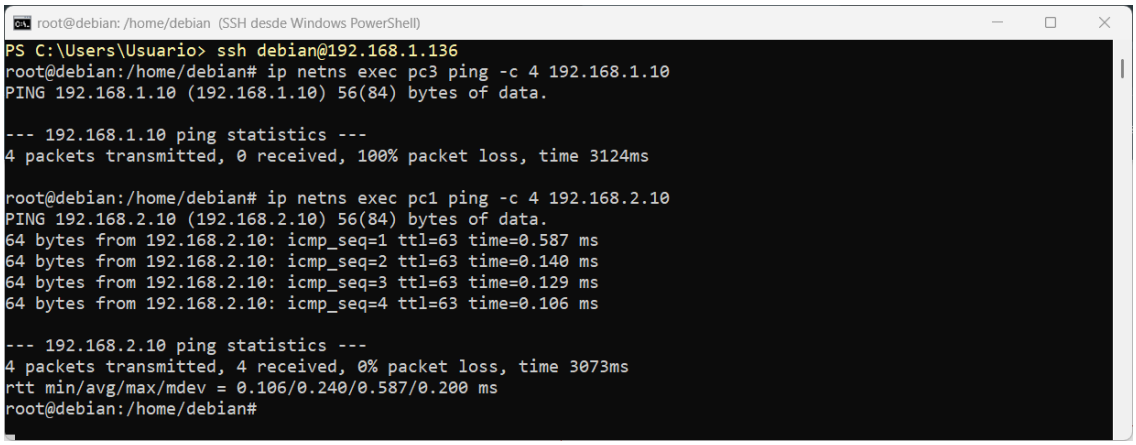


(b) Debian 13

Figura 11. Regla de firewall cargada en r1: (a) Ubuntu, (b) Debian. Fuente: elaboración propia.



(a) Ubuntu 24.04.4



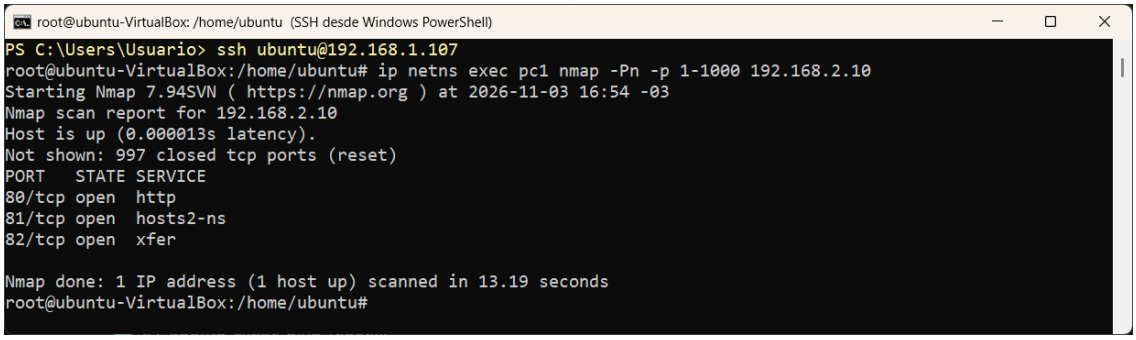
(b) Debian 13

Figura 12. Ping de pc3 a pc1 y de pc1 a pc3 con la regla activa: (a) Ubuntu, (b) Debian. Fuente: elaboración propia.

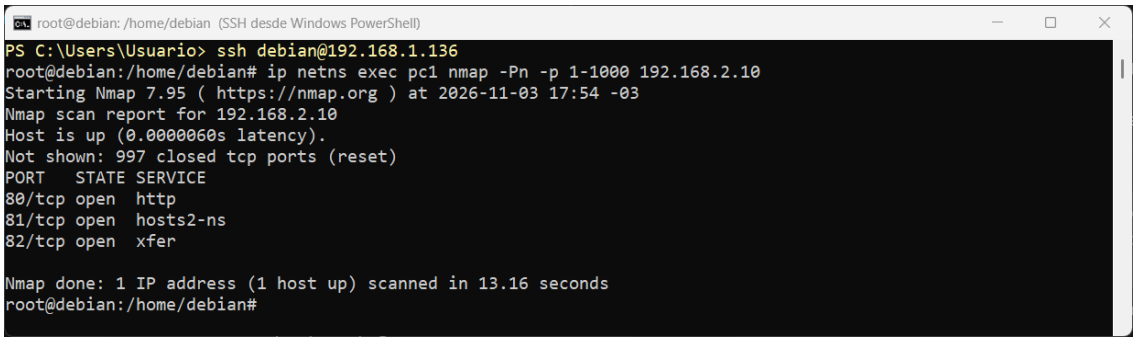
b) Escaneo de puertos con y sin firewall. Se levantan tres servicios de prueba en pc3, en los puertos 80, 81 y 82, y se escanean los puertos del 1 al 1000 desde pc1, primero sin firewall y después con una regla que descarta los puertos 81 y 82.

for p in 80 81 82; do ip netns exec pc3 python3 -m http.server $p >/dev/null 2>&1 & done
ip netns exec pc3 ss -tln
ip netns exec pc1 nmap -Pn -p 1-1000 192.168.2.10
ip netns exec r1 nft add table inet filtro2
ip netns exec r1 nft add chain inet filtro2 forward '{ type filter hook forward priority 0; policy accept; }'
ip netns exec r1 nft add rule inet filtro2 forward ip saddr 192.168.1.0/24 tcp dport 81-82 counter drop
ip netns exec pc1 nmap -Pn -p 1-1000 192.168.2.10

Sin firewall se esperan tres puertos abiertos. Con el firewall, el puerto 80 sigue abierto y los puertos 81 y 82 aparecen como filtrados [32]: de tres servicios, el atacante solo ve uno. También se anota el tiempo que informa nmap al final de cada escaneo.

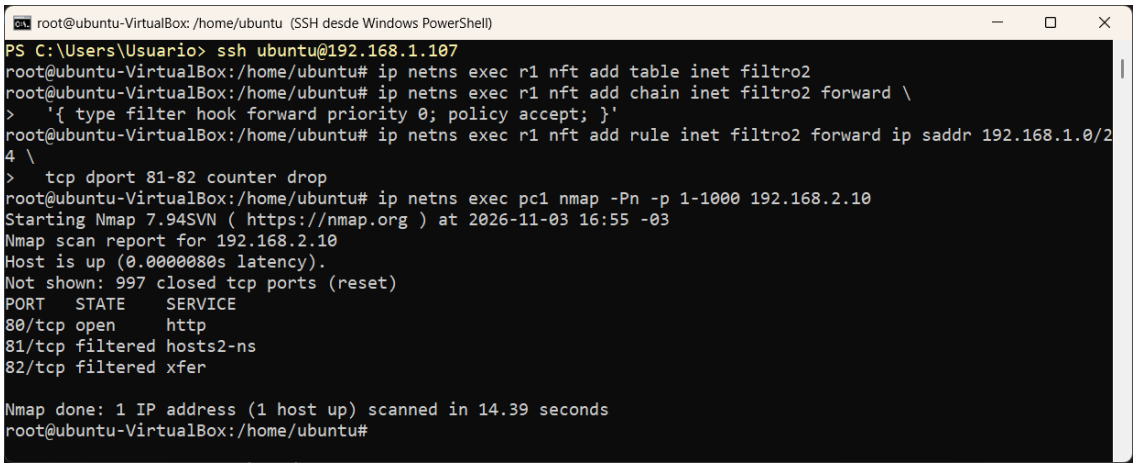


(a) Ubuntu 24.04.4

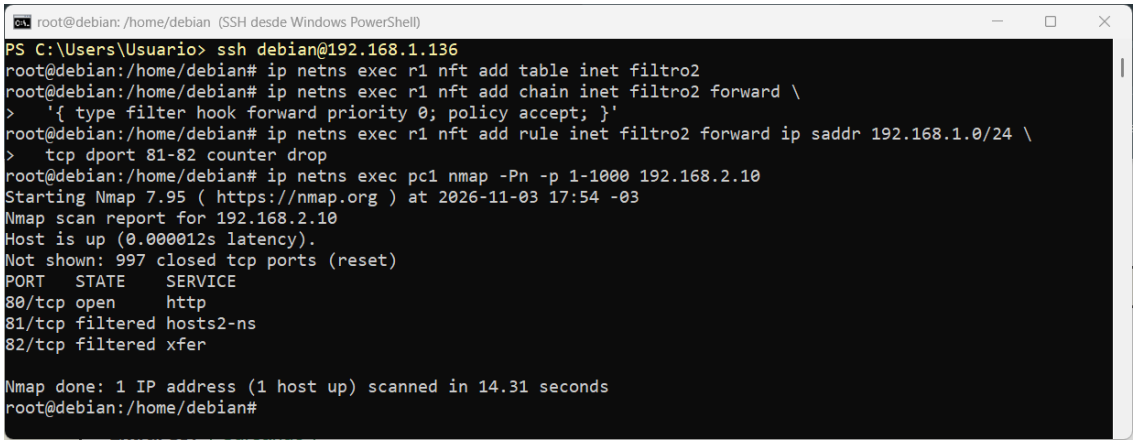


(b) Debian 13

Figura 13. Escaneo de puertos de pc3 sin firewall: (a) Ubuntu, (b) Debian. Fuente: elaboración propia.



(a) Ubuntu 24.04.4



(b) Debian 13

Figura 14. Escaneo de puertos de pc3 con firewall: (a) Ubuntu, (b) Debian. Fuente: elaboración propia.

4.7 Experimento 4: rendimiento del router

Se mide la velocidad de transferencia entre pc1 y pc3 a través del router, sin reglas y con 10 y con 100 reglas de nftables, para ver si la cantidad de reglas afecta al rendimiento. Es una prueba opcional. La velocidad se mide con iperf3 [33].

ip netns exec pc3 iperf3 -s -D
ip netns exec pc1 iperf3 -c 192.168.2.10 -t 5
ip netns exec r1 nft add table inet carga
ip netns exec r1 nft add chain inet carga forward '{ type filter hook forward priority 0; policy accept; }'
for i in $(seq 1 10); do ip netns exec r1 nft add rule inet carga forward ip saddr 10.99.0.$i drop; done
ip netns exec pc1 iperf3 -c 192.168.2.10 -t 5

for i in $(seq 11 100); do ip netns exec r1 nft add rule inet carga forward ip saddr 10.99.0.$i drop; done

ip netns exec pc1 iperf3 -c 192.168.2.10 -t 5

Las direcciones 10.99.0.x no existen en la topología, por lo que las reglas se evalúan pero nunca descartan paquetes. La medición se hace primero con 10 reglas y después se agregan 90 más (100 en total); cada caso se mide 5 veces. Como todo corre dentro de una máquina virtual, la velocidad depende de la CPU de la máquina virtual y no de una red real.

4.8 Experimento 5: comparación entre Debian y Ubuntu

Se comparan las dos distribuciones en dos partes, a partir de lo explicado en la sección 2.9.

Parte A: estado de cada distribución. Se ejecutan en las dos máquinas los mismos comandos de solo lectura:

uname -r
dpkg-query -f '.\n' -W | wc -l
df -h /
free -m
ls /etc/netplan
cat /etc/netplan/*.yaml
cat /etc/resolv.conf
systemctl is-active NetworkManager systemd-networkd systemd-resolved
ufw status
nft list ruleset
which sudo
nmcli general status

Parte B: configuración persistente. Se crea una interfaz virtual llamada dummy0, con una dirección IP, usando NetworkManager [23], [29]. Se mide el tiempo que lleva y se reinicia la máquina para comprobar si la configuración se mantiene.

time nmcli connection add type dummy ifname dummy0 con-name dummy0 ip4 10.10.10.1/24
ip -br addr show dummy0

Se espera que el kernel y las herramientas de las secciones anteriores se comporten igual, y que las diferencias estén en los archivos de configuración, el DNS y los programas incluidos, como indica la Tabla 3.

5. Resultados

En esta sección se presentan los resultados medidos. Cada tabla incluye el valor esperado según el marco teórico y el valor obtenido en cada distribución.

5.1 Verificación de la configuración

Equipo	Interfaz	IP esperada	MAC en Ubuntu	MAC en Debian
pc1	pc1-eth0	192.168.1.10/24	6a:88:12:c1:c7:f3	8e:08:b3:fd:a9:a0
r1	r1-eth0	192.168.1.1/24	e2:7b:f3:01:1d:47	02:fb:f5:96:86:b7
r1	r1-eth1	192.168.2.1/24	be:c3:38:bf:45:92	7a:70:51:c8:4e:28 (66:9b:a0:00:05:ce en los armados siguientes)
pc3	pc3-eth0	192.168.2.10/24	22:37:f2:5f:54:d0	62:22:93:d4:8d:72

Tabla 7. Interfaces, direcciones IP y MAC verificadas. Fuente: elaboración propia.

5.2 Armado de la topología: tiempo y memoria

Distribución	Tiempo de armado (s)	Tiempo de limpieza (s)	Memoria antes (MB)	Memoria después (MB)	Diferencia (MB)
Ubuntu (promedio de 5)	2,301	0,211	1066,6	1068,2	+1,6
Debian (promedio de 5)	1,592	0,156	1203,0	1205,8	+2,8

Tabla 8. Tiempo y memoria del armado de la topología. Fuente: elaboración propia.

5.3 Conectividad y latencia

Etapa	Esperado	Ubuntu	Debian
Misma red	0% de pérdida	0% (rtt 0,053/0,302/1,811 ms)	0% (rtt 0,042/0,132/0,218 ms)
Otra red, sin rutas	Falla (Network is unreachable)	Falla: «ping: connect: La red es inaccesible»	Falla: «ping: connect: La red es inaccesible»
Con rutas, sin reenvío	100% de pérdida	100% de pérdida (ip_forward = 0)	100% de pérdida (ip_forward = 0)
Con reenvío	0% de pérdida y TTL 63 [3]	0% de pérdida en 5 × 20 pings, TTL 63	0% de pérdida en 5 × 20 pings, TTL 63
Latencia con reenvío (rtt promedio, ms)	Menor a 1 ms en una sola máquina	0,388 (5 repeticiones: 0,145 a 0,917)	0,178 (5 repeticiones: 0,150 a 0,275)
Traceroute	2 saltos	2 saltos (192.168.1.1 → 192.168.2.10)	2 saltos (192.168.1.1 → 192.168.2.10)

Tabla 9. Resultados de conectividad y latencia. Fuente: elaboración propia.

5.4 Captura de tráfico

Se espera un intercambio ARP (2 paquetes) y 6 paquetes ICMP (3 pedidos y 3 respuestas). El ARP puede repetirse.

Distribución	Interfaz	Paquetes ARP	Paquetes ICMP	Total
Ubuntu	r1-eth1	2	6	8
Ubuntu	r1-eth0	2	6	8
Debian	r1-eth1	2	6	8
Debian	r1-eth0	2	6	8

Tabla 10. Paquetes capturados con tcpdump. Fuente: elaboración propia.

5.5 Firewall

Prueba	Esperado	Ubuntu	Debian
Ping de pc3 a pc1, con la regla	100% de pérdida	100% de pérdida	100% de pérdida
Ping de pc1 a pc3, con la regla	0% de pérdida	0% de pérdida	0% de pérdida
Ping de pc3 a pc1, sin la regla	0% de pérdida	0% de pérdida	0% de pérdida
Paquetes descartados (contador)	4	4 (336 bytes)	4 (336 bytes)

Tabla 11. Resultados del bloqueo de ping con nftables. Fuente: elaboración propia.

Escaneo (puertos 1 a 1000)	Esperado	Ubuntu	Debian
Puertos abiertos, sin firewall	3	3 (80, 81, 82)	3 (80, 81, 82)
Puertos abiertos, con firewall	1	1 (80)	1 (80)
Puertos filtrados, con firewall	2	2 (81, 82)	2 (81, 82)
Tiempo del escaneo sin firewall (s)	–	13,19	13,16
Tiempo del escaneo con firewall (s)	–	14,39	14,31
Paquetes descartados (contador)	–	4 (176 bytes)	4 (176 bytes)

Tabla 12. Resultados del escaneo de puertos con y sin firewall. Fuente: elaboración propia.

5.6 Rendimiento del router

Se espera que la velocidad se mantenga estable o baje levemente al aumentar la cantidad de reglas. Resultado: en Ubuntu la velocidad bajó un 21 % con 100 reglas, más que «levemente». En Debian no siguió una tendencia, porque la variación entre repeticiones (7,9 a 11,7 Gbit/s) fue mayor que el efecto de las reglas. Con 5 repeticiones de 5 s, la diferencia entre distribuciones (alrededor del 25 % a favor de Debian) es más clara que el efecto de las reglas.

Reglas en r1	Ubuntu (Mbit/s)	Debian (Mbit/s)
0	7840 (7,84 Gbit/s)	9840 (9,84 Gbit/s)
10	7270 (7,27 Gbit/s)	10750 (10,75 Gbit/s)
100	6170 (6,17 Gbit/s)	8930 (8,93 Gbit/s)

Tabla 13. Velocidad de transferencia según la cantidad de reglas. Fuente: elaboración propia.

5.7 Comparación entre Debian y Ubuntu

Dato	Ubuntu	Debian
Versión del kernel	7.0.0-42-generic	6.12.107+deb13-amd64
Paquetes instalados	1522	2390

Dato	Ubuntu	Debian
Espacio usado en disco	9,6 GB de 25 GB (42 %)	11 GB de 19 GB (58 %)
RAM usada en reposo (MB)	1072	1108
Archivos de configuración de red	/etc/netplan/ (01-network-manager-all.yaml, 50-cloud-init.yaml). NetworkManager guarda sus conexiones como /etc/netplan/90-NM-<uuid>.yaml	/etc/netplan no existe. NetworkManager guarda sus conexiones en /etc/NetworkManager/system-connections/*.nmconnection
Resolución de nombres (DNS)	systemd-resolved activo (nameserver 127.0.0.53)	resolv.conf generado por NetworkManager (nameserver 192.168.1.1); systemd-resolved inactivo
Firewall incluido	ufw instalado e inactivo; ruleset de nftables vacío	ufw no instalado («orden no encontrada»); ruleset de nftables vacío
Tiene sudo	Sí (/usr/bin/sudo); root con sudo -i	Sí (/usr/bin/sudo); root con su -
¿La IP se mantiene al reiniciar?	Sí (dummy0 10.10.10.1/24)	Sí (dummy0 10.10.10.1/24)
Tiempo de aplicar la configuración (s)	2,230	0,115

Tabla 14. Comparación del estado de cada distribución. Fuente: elaboración propia.

5.8 Resumen y problemas encontrados

De 23 pruebas con un resultado esperado definido (7 namespaces, 4 direcciones IP, 6 etapas de conectividad, 4 capturas de tráfico, 4 pruebas de bloqueo de ping, 3 conteos de puertos y la persistencia de dummy0), las 23 dieron el resultado esperado en las dos distribuciones: 46 de 46 verificaciones. Lo que no coincidió del todo con lo esperado fue: (1) el rendimiento con reglas (sección 5.6): en Ubuntu la velocidad bajó un 21 % con 100 reglas (de 7840 a 6170 Mbit/s), más que «levemente», y en Debian no siguió una tendencia (9840, 10750 y 8930 Mbit/s), porque la variación entre repeticiones (7,9 a 11,7 Gbit/s) fue mayor que el efecto de las reglas; (2) en la captura sobre r1-eth0 el primer echo-request aparece antes del ARP, porque r1 recién pregunta por la MAC de pc1 cuando debe enviarle la respuesta; (3) el mensaje de error se muestra en español («La red es inaccesible») porque el sistema está en ese idioma.

#	Problema (mensaje de error)	Distribución	Causa	Solución	Tiempo (min)
1	Ventana «Actualización de software» tapó la pregunta de iperf3 durante apt install	Ubuntu	update-notifier con 128 actualizaciones pendientes	Volver a la Terminal (Alt+Tab) y responder «No»	1
2	«Cannot create namespace file "/run/netns/pc1": File exists» (prueba intencional)	Ambas	La topología ya estaba armada	bash limpiar.sh y armar de nuevo	0

#	Problema (mensaje de error)	Distribución	Causa	Solución	Tiempo (min)
3	La VM estaba en estado «guardado» y no se podía cambiar su configuración	Debian	Quedó suspendida en una sesión anterior	Reanudarla, apagarla con systemctl poweroff y después modifyvm	1
4	Arranque muy lento: pantalla negra con avisos de vmwgfx (systemd-analyze: 5 min 31 s y 8 min 43 s)	Debian	Fase de kernel muy larga dentro de la VM (5 min 5 s y 8 min 17 s); userspace 25-26 s	Esperar; se registra como dato (Ubuntu arrancó en menos de 1 min)	15
5	Teclas perdidas y una tecla repitiéndose al escribir en la Terminal	Debian	La VM estaba ocupada con la animación de maximizar la ventana	Ctrl + C y volver a escribir más despacio	1
6	«ufw: orden no encontrada»	Debian	ufw no viene instalado	Ninguna: es un dato para la Tabla 14	0
7	Después de nmcli connection delete dummy0, nmcli connection show todavía la listaba	Ambas	NetworkManager actualizando su estado	Se verificó que el archivo de configuración se borró	0,5
8	La MAC de r1-eth1 cambió entre armados (7a:70:51:c8:4e:28 → 66:9b:a0:00:05:ce)	Debian	Las veth reciben una MAC generada al crearse	Usar en cada análisis la MAC de su captura	0,5

Tabla 15. Problemas encontrados durante las pruebas. Fuente: elaboración propia.

6. Conclusiones

Respuesta a la pregunta. Sí: como router y firewall, Debian y Ubuntu se comportaron igual. Con el reenvío activado, las dos tuvieron 0 % de pérdida en 100 pings (5 × 20), con TTL 63 y 2 saltos; sin reenvío, 100 % de pérdida. La regla de nftables descartó exactamente 4 de 4 pedidos de ping en ambas, y con el firewall un atacante pasó de ver 3 puertos abiertos a ver solo 1 (81 y 82 filtrados) en las dos. Las diferencias aparecieron en el rendimiento y en la forma de configurar la red, no en el comportamiento.

Qué se comprobó. El marco teórico indica que el enrutamiento lo realiza el kernel, que un equipo Linux solo reenvía paquetes entre redes si se activa el reenvío [13], y que Netfilter filtra el tráfico que atraviesa el router [18]. Los resultados lo confirman: sin ruta, el kernel rechaza el envío («La red es inaccesible»); con ip_forward en 0, el router descarta el 100 % de los paquetes, y al activarlo pasan todos. tcpdump mostró que las direcciones IP se mantienen de punta a punta (192.168.1.10 → 192.168.2.10) mientras las MAC cambian en cada red, y Netfilter, en el hook FORWARD, descartó 4 de 4 echo-request y los paquetes dirigidos a los puertos 81 y 82. Armar los 7 namespaces llevó 2,30 s en Ubuntu y 1,59 s en Debian, con menos de 3 MB de memoria adicional, lo que confirma que son livianos.

Diferencias entre distribuciones. Se esperaban diferencias en la forma de configurar la red, en el DNS y en el firewall incluido (Tabla 3). Se confirmaron: Ubuntu guarda la configuración de NetworkManager a través de Netplan (/etc/netplan/90-NM-<uuid>.yaml) y resuelve nombres con systemd-resolved (127.0.0.53); Debian no tiene /etc/netplan, guarda las conexiones en /etc/NetworkManager/system-connections/ y su /etc/resolv.conf lo escribe NetworkManager. Ubuntu trae ufw (inactivo) y Debian no. Además aparecieron diferencias numéricas: Debian armó la topología un 31 % más rápido (1,59 s frente a 2,30 s), tuvo menor latencia (0,178 ms frente a 0,388 ms), mayor velocidad (9840 frente a 7840 Mbit/s sin reglas) y aplicó la configuración persistente en 0,115 s frente a 2,230 s. Ubuntu, en cambio, instala menos paquetes (1522 frente a 2390) y arrancó en menos de un minuto, mientras que Debian tardó entre 5 y 9 minutos en esta máquina virtual.

Distribución más adecuada para el trabajo realizado. Para el trabajo realizado (una topología de namespaces con un router Linux y un firewall con nftables), las dos distribuciones cumplen el 100 % de las pruebas funcionales, por lo que cualquiera sirve. Si se prioriza el rendimiento y la liviandad del entorno de red, Debian resultó más adecuada: armó la topología en 1,59 s (Ubuntu: 2,30 s), tuvo la mitad de latencia (0,178 ms frente a 0,388 ms), alrededor de un 25 % más de velocidad a través del router (9840 frente a 7840 Mbit/s sin reglas) y aplicó la configuración persistente casi 20 veces más rápido (0,115 s frente a 2,230 s), con una configuración más directa (keyfiles de NetworkManager y resolv.conf sin intermediarios). Ubuntu, en cambio, fue más práctica de usar: trae sudo y ufw listos, resuelve nombres con caché (systemd-resolved) y arrancó en menos de un minuto, mientras que la VM de Debian tardó entre 5 y 9 minutos. Como el kernel es distinto en cada una (6.12 en Debian y 7.0 en Ubuntu) y cada medición se repitió solo cinco veces, las diferencias de rendimiento deben tomarse como una tendencia y no como una conclusión definitiva.

Limitaciones.

- Todo se ejecutó dentro de una máquina virtual, por lo que la velocidad medida depende de la CPU de la máquina virtual y no de una red real.

- Cada medición se repitió cinco veces, lo que alcanza para comparar pero no para un análisis estadístico.

- Los resultados valen para las versiones y los recursos utilizados.

Trabajo futuro. A partir de este trabajo se podrían probar NAT con masquerade, el filtrado con estado con conntrack, una VPN con WireGuard o el enrutamiento dinámico con FRRouting.

Referencias

Libros

[1] Nemeth, E., Snyder, G., Hein, T. R., Whaley, B., Mackin, D.: UNIX and Linux System Administration Handbook. 5.ª ed. Addison-Wesley, Boston (2017).

[2] Tanenbaum, A. S., Wetherall, D. J.: Redes de computadoras. 5.ª ed. Pearson Educación, México (2012).

RFC (estándares de protocolos)

[3] Postel, J.: RFC 791, Internet Protocol (1981). Disponible en: https://www.rfc-editor.org/rfc/rfc791

[4] Postel, J.: RFC 792, Internet Control Message Protocol (1981). Disponible en: https://www.rfc-editor.org/rfc/rfc792

[5] Plummer, D.: RFC 826, An Ethernet Address Resolution Protocol (1982). Disponible en: https://www.rfc-editor.org/rfc/rfc826

[6] Postel, J.: RFC 768, User Datagram Protocol (1980). Disponible en: https://www.rfc-editor.org/rfc/rfc768

[7] Eddy, W. (ed.): RFC 9293, Transmission Control Protocol (TCP) (2022). Disponible en: https://www.rfc-editor.org/rfc/rfc9293

Documentación de Linux y herramientas

[8] Ubuntu Manpage Repository: ip-link(8). Disponible en: https://manpages.ubuntu.com/manpages/noble/man8/ip-link.8.html

[9] systemd: Predictable Network Interface Names. Disponible en: https://systemd.io/PREDICTABLE_INTERFACE_NAMES/

[10] Rekhter, Y., Moskowitz, B., Karrenberg, D., de Groot, G. J., Lear, E.: RFC 1918, Address Allocation for Private Internets (1996). Disponible en: https://www.rfc-editor.org/rfc/rfc1918

[11] Fuller, V., Li, T.: RFC 4632, Classless Inter-domain Routing (CIDR) (2006). Disponible en: https://www.rfc-editor.org/rfc/rfc4632

[12] man7.org: ip-route(8). Disponible en: https://man7.org/linux/man-pages/man8/ip-route.8.html

[13] The Linux Kernel Documentation: IP Sysctl. Disponible en: https://docs.kernel.org/networking/ip-sysctl.html

[14] Ubuntu Manpage Repository: hosts(5), resolv.conf(5), nsswitch.conf(5). Disponible en: https://manpages.ubuntu.com/manpages/noble/man5/hosts.5.html, https://manpages.ubuntu.com/manpages/noble/man5/resolv.conf.5.html y https://manpages.ubuntu.com/manpages/noble/man5/nsswitch.conf.5.html

[15] Mockapetris, P.: RFC 1034, Domain Names – Concepts and Facilities; RFC 1035, Domain Names – Implementation and Specification (1987). Disponible en: https://www.rfc-editor.org/rfc/rfc1034 y https://www.rfc-editor.org/rfc/rfc1035

[16] Droms, R.: RFC 2131, Dynamic Host Configuration Protocol (1997). Disponible en: https://www.rfc-editor.org/rfc/rfc2131

[17] systemd: systemd-resolved.service(8). Disponible en: https://www.freedesktop.org/software/systemd/man/latest/systemd-resolved.service.html

[18] The netfilter.org project. Disponible en: https://www.netfilter.org/

[19] nftables wiki. Disponible en: https://wiki.nftables.org/

[20] Ubuntu Security Documentation: Firewall. Disponible en: https://documentation.ubuntu.com/security/security-features/network/firewall/

[21] Ubuntu Manpage Repository: ip(8), ss(8), ping(8), traceroute(1), tcpdump(8). Disponible en: https://manpages.ubuntu.com/manpages/noble/man8/ip.8.html, https://manpages.ubuntu.com/manpages/noble/man8/ss.8.html, https://manpages.ubuntu.com/manpages/noble/man8/ping.8.html, https://manpages.ubuntu.com/manpages/noble/man1/traceroute.db.1.html y https://manpages.ubuntu.com/manpages/noble/man8/tcpdump.8.html

[22] Red Hat Blog: Linux networking: ifconfig vs ip. Disponible en: https://www.redhat.com/es/blog/ifconfig-vs-ip

[23] Red Hat: Red Hat Enterprise Linux 9 – Configuring and managing networking. Disponible en: https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/9/html/configuring_and_managing_networking/configuring-an-ethernet-connection_configuring-and-managing-networking

[24] Netplan documentation. Disponible en: https://netplan.io/

[25] Ubuntu Manpage Repository: network_namespaces(7), ip-netns(8). Disponible en: https://manpages.ubuntu.com/manpages/noble/man7/network_namespaces.7.html y https://manpages.ubuntu.com/manpages/noble/man8/ip-netns.8.html

[26] Mininet. Disponible en: https://mininet.org/

Documentación de distribuciones, herramientas y virtualización

[27] Debian Wiki: NetworkConfiguration. Disponible en: https://wiki.debian.org/NetworkConfiguration

[28] Hertzog, R., Mas, R.: The Debian Administrator's Handbook, sección Configuring the Network. Disponible en: https://www.debian.org/doc/manuals/debian-handbook/sect.network-config.en.html

[29] Debian Wiki: NetworkManager. Disponible en: https://wiki.debian.org/NetworkManager

[30] Debian Wiki: Derivatives/Census/Ubuntu. Disponible en: https://wiki.debian.org/Derivatives/Census/Ubuntu

[31] Ubuntu Community Help Wiki: UFW. Disponible en: https://help.ubuntu.com/community/UFW

[32] Nmap Project: Nmap Reference Guide. Disponible en: https://nmap.org/book/man.html

[33] ESnet: iperf3 user documentation. Disponible en: https://software.es.net/iperf/

[34] Oracle: Oracle VM VirtualBox User Guide, Virtual Networking. Disponible en: https://docs.oracle.com/en/virtualization/virtualbox/7.0/user/networkingdetails.html